

Um estudo multinacional sobre leitura e Habilidades de rastreamento em programadores iniciantes

Raymond Lister Faculdade
de Tecnologia da Informação Universidade
de Tecnologia de Sydney Broadway, NSW
2007, Austrália +61 2 9514 1850
raymond@it.uts.edu.au

Elizabeth S. Adams
Departamento de Ciência da
Computação Universidade
James Madison Harrisonburg, VA
22807, EUA +1 (540)
568 1667 adamses@jmu.edu

Sue Fitzgerald,
Ciências da Informação e da Computação,
Universidade Estadual
Metropolitana, St. Paul, MN
55106, EUA, +1 (651)
793-1473, sue.fitzgerald@metrostate.edu

William Fone
Universidade de Staffordshire
Stafford, ST18 0DG
Reino Unido
+44-1785-353304
W.Fone@Staffs.ac.uk

João Hamer
Departamento de Ciência da Computação
Universidade de Auckland
Auckland, Nova Zelândia +64 9
3737 599
j.hamer@cs.auckland.ac.nz

Morten Lindholm
Departamento de Ciência da Computação
Universidade de Aarhus
Aarhus, DK-8200, Dinamarca +45
8942 5621
lindholm@daimi.au.dk

Robert McCartney
Departamento de Ciência da
Computação e Engenharia
Universidade de
Connecticut Storrs,
CT 06268 EUA +1 (860) 486-5232 robert@cse.uconn.edu

Jan Erik Moström
Departamento de Ciência da Computação
Universidade de Umeå
905 86 Umeå, Suécia +46
90 147 289
jan@cs.umu.se

Kate Sanders
Departamento de
Matemática/Ciência da
Computação Rhode Island College
Providence, RI 02908
EUA +1 (401) 456-9634 ksanders@ric.edu

Otto Seppälä
Laboratório de Ciência de Processamento de
Informação Universidade de Tecnologia
de Helsinque PO Box 5400, 02015
TKK, Finlândia +358
(9)451 5193 oseppala@cs.hut.fi

Beth Simon
Departamento de Matemática e Ciência da
Computação Universidade
de San Diego San Diego,
CA 92110 +1 (619)
260-7929 bsimon@sandiego.edu

Lynda Thomas
Departamento de Ciência da
Computação
Universidade
de Gales Aberystwyth
+44 (1970) 622452 ltt@aber.ac.uk

RESUMO Um

estudo realizado por um grupo de trabalho do ITiCSE 2001 ("o Grupo McCracken") constatou que muitos alunos não sabem programar ao final de seus cursos introdutórios. Uma explicação comum para essa incapacidade é a falta de habilidade para resolver problemas. Ou seja, os alunos não conseguem pegar a descrição de um problema, decompô-lo em subproblemas e implementá-los, para então montar as partes em uma solução completa. Uma explicação alternativa é que muitos alunos têm uma compreensão frágil tanto dos princípios básicos de programação quanto da capacidade de executar sistematicamente tarefas rotineiras de programação, como rastrear (ou "verificar" o código). Este grupo de trabalho do ITiCSE 2004 estudou a explicação alternativa, testando alunos de sete países de duas maneiras. Primeiro, os alunos foram testados em sua capacidade de prever o resultado da execução de um pequeno trecho de código. Segundo, os alunos foram testados em sua capacidade de, ao receberem a função desejada de um pequeno trecho de código, executar o código. de um código quase completo, para selecionar a conclusão correta do código a partir de um pequeno conjunto de possibilidades. Muitos alunos apresentaram dificuldades nessas tarefas, especialmente na última, sugerindo que esses alunos têm um domínio frágil de habilidades que são pré-requisitos para a resolução de problemas.

1. INTRODUÇÃO Apesar dos

melhores esforços dos professores em nossa disciplina, muitos alunos ainda enfrentam dificuldades com programação. Um grupo de trabalho do ITICSE de 2001 (o “grupo McCracken”) avaliou a capacidade de programação de uma grande população de estudantes de diversas universidades, nos Estados Unidos e em outros países [McCracken, 2001]. Os autores testaram os alunos em um conjunto comum de problemas de programação. A maioria dos alunos teve um desempenho muito pior do que o esperado. De fato, a maioria dos alunos sequer chegou perto de concluir a tarefa proposta. Os resultados são convincentes, dada a natureza multinacional da colaboração. Enquanto um relatório semelhante de um autor em uma única instituição poderia ser descartado como consequência do ensino deficiente nessa instituição, é difícil descartar um estudo multinacional.

Embora o trabalho do grupo McCracken tenha destacado a extensão do problema, a natureza do estudo de McCracken não isola as causas do problema. Uma explicação comum para o baixo desempenho dos alunos é a falta de capacidade de resolução de problemas. O grupo McCracken definiu a resolução de problemas como um processo de cinco etapas: (1) Abstrair o problema a partir de sua descrição, (2)

Gerar subproblemas, (3) Transformar subproblemas em subsoluções, (4) Recompôr e (5) Avaliar e iterar. No entanto, existem outras explicações possíveis para as dificuldades dos alunos em programar. Por exemplo, os alunos podem simplesmente não entender as construções de programação necessárias para produzir um programa (como vetores ou recursão). Outra explicação, mais sutil, é que o conhecimento do aluno é “frágil”. Ou seja, embora um aluno possa articular itens específicos de conhecimento quando explicitamente solicitado, quando lhe pedem para aplicar esse conhecimento em um contexto de escrita de programa, ele “meio que sabe, tem alguns fragmentos, consegue fazer alguns movimentos, tem uma noção, sem ser capaz de reunir conhecimento suficiente com precisão para levar um problema a uma solução adequada” [Perkins e Martin, 1986, p. 214]. Neste artigo, usamos o termo “conhecimento frágil” para incluir também habilidades básicas, como a capacidade de executar (“rastrear”) um trecho de código de forma sistemática e manual.

Como Perkins e Martin apontam, a resolução de problemas em geral e o conhecimento não são “duas dimensões independentes da programação” [p. 226]. Tampouco a aquisição abrangente de conhecimento e habilidades de programação é um pré-requisito absoluto para manifestar a capacidade de resolver problemas. No entanto, alguma compreensão mínima dos conceitos de programação e habilidades associadas é necessária para que um aluno possa manifestar habilidades de resolução de problemas no sentido estrito de cinco etapas, conforme definido pelo grupo McCracken. Essas considerações nos levaram a fazer a seguinte pergunta: em que medida os alunos tiveram um desempenho ruim na

O estudo de McCracken se deve a habilidades deficientes de resolução de problemas ou a conhecimentos e habilidades frágeis que são precursores da resolução de problemas?

Este artigo é o relatório de um grupo de trabalho do ITICSE 2004, que explorou a questão acima, pedindo aos alunos que demonstrassem sua compreensão de códigos existentes. Se um aluno consegue demonstrar consistentemente a compreensão de códigos existentes, mas tem dificuldades para escrever programas semelhantes, então pode ser razoável concluir que ele não possui as habilidades necessárias para a resolução de problemas.

No entanto, se um aluno não consegue demonstrar consistentemente compreensão do código existente, então a dificuldade desse aluno reside na falta de conhecimento e habilidades que são pré-requisitos para a resolução de problemas complexos, que envolvem cinco etapas. Os dados para este grupo de trabalho foram coletados por meio de doze questões de múltipla escolha respondidas pelos alunos. O conjunto completo de questões de múltipla escolha encontra-se no Apêndice A. Nesse apêndice, o código de programação está em Java. No entanto, as questões visam testar o conhecimento e as habilidades dos alunos em processos iterativos genéricos com arrays e, portanto, podem ser facilmente traduzidas para diversas linguagens de programação. Dentro do grupo de trabalho, uma das instituições participantes traduziu as questões de múltipla escolha para outra linguagem de programação, o C++.

Neste estudo, utilizamos questões de múltipla escolha como instrumento de avaliação dos alunos por dois motivos. Primeiro, por se tratar de um estudo multi-institucional, buscávamos uma forma de pontuar o desempenho dos alunos nas questões de múltipla escolha que não exigisse julgamento subjetivo por parte de cada membro do grupo de trabalho. Em segundo lugar, estávamos preocupados com a possibilidade de que, se os alunos fossem obrigados a explicar a função de um trecho de código, o baixo desempenho pudesse ser atribuído à falta de eloquência, e não à falta de compreensão do código (especialmente quando o aluno estivesse sendo entrevistado em um idioma diferente de sua língua materna).

2. COLETA DE DADOS Ao responder às

12 questões de múltipla escolha, os alunos forneceram três tipos de dados, descritos nas três subseções seguintes.

2.1 Dados de desempenho O

objetivo da coleta desses dados foi avaliar a dificuldade das questões de múltipla escolha em todas as instituições participantes.

Cada membro do grupo de trabalho testou os alunos com base nas doze questões de múltipla escolha, em condições de exame. Os dados primários coletados foram as respostas dos alunos para cada questão, a partir das quais foi possível calcular uma pontuação de 0 a 12. Um total de 941 alunos contribuiu com dados para esta parte do estudo, mas desses alunos, apenas 556 responderam a todas as doze questões.

A maioria dos alunos que realizaram este teste de desempenho havia concluído recentemente, ou estava prestes a concluir, o primeiro semestre de seus estudos em programação. No entanto, em diversas instituições, muitos alunos não estavam no primeiro semestre de seus estudos, ou mesmo no primeiro ano.

Em algumas instituições, as questões de múltipla escolha foram utilizadas como parte do procedimento para atribuir a nota final aos alunos. Em outras instituições, os alunos se voluntariaram para participar do estudo.

2.2 Transcrições das Entrevistas O

objetivo da coleta desses dados foi investigar como os alunos responderam às questões de múltipla escolha.

Cada um dos doze membros do grupo de trabalho entrevistou pelo menos três alunos. No total, 37 alunos foram entrevistados. Durante a entrevista, os alunos foram solicitados a "pensar em voz alta" enquanto respondiam ao conjunto principal de questões de múltipla escolha. As entrevistas foram gravadas e posteriormente transcritas. Alguns alunos foram entrevistados em um idioma diferente do inglês. Nesses casos, a versão final da transcrição foi uma tradução da entrevista para o inglês.

Nas instituições onde as doze questões de múltipla escolha foram utilizadas como parte do processo de avaliação, as entrevistas foram realizadas após os alunos terem feito o teste. Nesses casos, a entrevista serviu como um debriefing, na qual os alunos relembrou como responderam a cada questão. Em outras entrevistas, onde as questões de múltipla escolha não faziam parte do processo de avaliação, os alunos viram as questões pela primeira vez durante a entrevista. A resposta escolhida por cada questão de múltipla escolha também foi incluída nos dados de desempenho discutidos na seção 2.1.

Entre os entrevistados, a idade mediana foi de 20 anos, variando de 18 a 54 anos. Dez por cento eram mulheres. O tempo médio de permanência nas respectivas instituições era de 1,4 anos, com um desvio padrão de 0,9. A maioria dos alunos estava em seu primeiro curso de programação. Os alunos de Java estudavam programação há uma média de 0,45 anos, e os de C++, uma média de 0,3 anos.

2.3 Dados do Doodle O

objetivo da coleta desses dados foi investigar como os alunos responderam às questões de múltipla escolha.

Os alunos receberam papel para rascunho, no qual podiam desenhar ou fazer cálculos para responder às questões de múltipla escolha. Dentro do grupo de trabalho, esses desenhos e cálculos são chamados de "rabiscos". Normalmente, o aluno podia rabiscar na mesma página em que a questão era apresentada.

Foram coletados desenhos de todos os 37 alunos entrevistados. Em algumas instituições, os desenhos também foram coletados de alunos que forneceram dados de desempenho.

3. ANÁLISE DOS DADOS DE DESEMPENHO Esta seção contém uma análise estatística dos dados de desempenho, sem qualquer referência detalhada à transcrição ou aos dados do Doodle.

A Figura 3.1 mostra um histograma das pontuações (em uma escala de 12) dos 556 alunos que responderam às doze questões. As pontuações abrangem toda a gama, de 0 a 12, com moda de 8. A Tabela 3.1 mostra os limites dos quartis para esses alunos. Esses limites de quartis são amplamente utilizados na análise dos dados de desempenho desta seção.

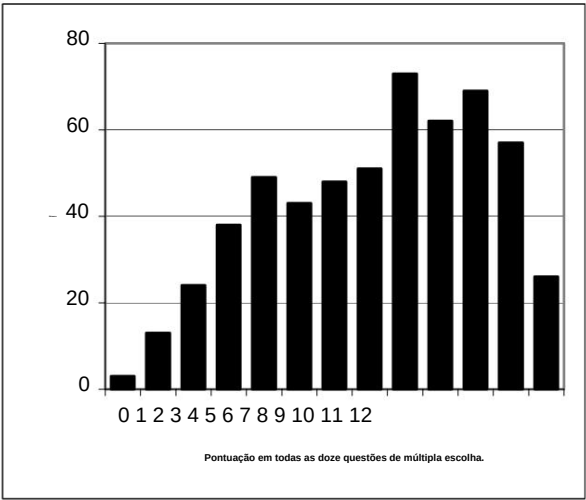


Figura 3.1. Distribuição das pontuações dos alunos que responderam a todas as 12 questões de múltipla escolha (N=556).

Quartil	Pontuação Faixa	Número de alunos	Porcentagem de Estudantes
1º "topo"	10 - 12	152	27%
2º	8 - 9	135	24%
	5 - 7	142	25%
3º 4º "fundo"	0 - 4	127	23%

Tabela 3.1. Os limites dos quartis para os alunos que responderam a todas as 12 questões de múltipla escolha (N=556).

Quartil	Pontuação Faixa	Número de alunos	Porcentagem de Estudantes
1º "topo"	10 - 12	46	20%
2º	8 - 9	56	25%
	5 - 7	60	26%
3º 4º "fundo"	0 - 4	66	29%

Tabela 3.2. Os limites dos quartis para os alunos que responderam a todas as 12 questões de múltipla escolha, de todas as instituições, exceto uma (N=228).

Uma única instituição contribuiu com mais da metade dos dados de desempenho resumidos na Tabela 3.1. (A segunda maior contribuição de uma única instituição foi de 11%. Seis instituições contribuíram com dados para todas as 12 questões de múltipla escolha, com pelo menos 20 alunos.) Portanto, existe a possibilidade de que os limites dos quartis na Tabela 3.1 sejam indevidamente influenciados pelos alunos da instituição que contribuiu com mais da metade dos dados de

desempenho. A Tabela 3.2 mostra uma distribuição do número de alunos, pelos limites dos quartis estabelecidos na Tabela 3.1, mas excluindo os alunos da instituição que contribuiu com a maioria dos dados de desempenho. A Tabela 3.2 mostra que a porcentagem de alunos de todas as outras instituições no quartil superior é menor (20% em comparação com 27% na Tabela 3.1) e a porcentagem de alunos no quartil inferior é maior (29% em comparação com 23% na Tabela 3.1). A porcentagem de alunos nos dois quartis intermediários é quase a mesma. Com base nessa comparação, e para os fins deste estudo, concluímos que as estatísticas de desempenho dos alunos são influenciadas, mas não dominadas, por essa única instituição. Na medida em que as estatísticas de desempenho são influenciadas por essa única instituição, o efeito é o de aumentar as notas dos alunos na população total.

As estatísticas de desempenho acima não são tão drasticamente ruins quanto os números do estudo de McCracken, onde mais da metade dos alunos teve um desempenho muito fraco. Esperávamos que a distribuição de desempenho em nosso estudo fosse melhor do que a distribuição no estudo de McCracken, dada a nossa suposição de que a capacidade de ler e compreender pequenos trechos de código é uma habilidade pré-requisito para a resolução de problemas em cinco etapas. No entanto, é difícil determinar o que é uma pontuação "boa" nas 12 questões de múltipla escolha sem primeiro estudá-las. Por esse motivo, as duas subseções seguintes examinam cada uma uma questão de múltipla escolha específica. Uma análise semelhante das dez questões de múltipla escolha restantes é apresentada no Apêndice B. As subseções subsequentes examinam os dados estatísticos de todas as doze questões de múltipla escolha.

3.1 Questão 2: Uma questão de múltipla escolha de nível médio. A Figura 3.2 contém uma questão de múltipla escolha, a Questão 2, do conjunto completo de 12 questões. Ela foi respondida corretamente por 65% de todos os alunos que a tentaram e é considerada de nível médio no conjunto de 12 questões.

À primeira vista, a questão 2 pode parecer contar o número de elementos comuns em ambos os arrays, que é 3 (alternativa A). No entanto, analisando o código mais detalhadamente, percebe-se que os elementos na posição 0 dos arrays não são contados, portanto a resposta correta é 2 (alternativa B).

Ignorar o primeiro elemento de um array não é idiomático – não é o que muitos programadores esperariam – mas pode estar correto. Por exemplo, o primeiro elemento de um array em C ou C++ às vezes é usado para armazenar o comprimento do array, em vez de um elemento do array. Além disso, muitos bugs são causados pelo fato de o código que escrevemos não o fazer.

Nem sempre o código faz o que pretendemos, portanto, a capacidade de ler o que o código realmente faz, em vez do que achamos que ele deveria fazer, é uma habilidade de programação importante. Independentemente de a Questão 2 ser um trecho de código que um professor deva mostrar à sua turma, certamente é um trecho de código que um aluno pode escrever e precisar depurar.

Questões de múltipla escolha (QMC) são uma forma comum de avaliar alunos em diversas disciplinas, e existe um corpo considerável de literatura dedicado à construção e análise desses testes [Ebel & Frisbie, 1986; Linn & Gronlund, 1995; Haladyna, 1999]. Uma maneira comum de analisar a eficácia de uma QMC baseia-se na noção de que a maioria dos alunos com bom desempenho deve respondê-las corretamente, enquanto a maioria dos alunos com baixo desempenho deve respondê-las incorretamente. Para a Questão 2, aproximadamente 90% dos alunos no primeiro quartil (ou seja, alunos que obtiveram pontuação entre 10 e 12 em todas as 12 QMC) responderam corretamente, enquanto aproximadamente 30% dos alunos no quartil inferior (ou seja, alunos que obtiveram pontuação entre 0 e 4 em todas as 12 QMC) responderam corretamente. (Note que um aluno que não entende nada sobre a questão e simplesmente chuta tem 25% de chance de acertar, enquanto um aluno que consegue eliminar uma opção tem 33% de chance de acertar.) Com base nessas duas porcentagens para os quartis superior e inferior, esta questão de múltipla escolha distingue entre alunos mais fortes e mais fracos.

Uma análise de quartis semelhante, porém mais abrangente, da Questão 2 é apresentada na Figura 3.3. Esse tipo de figura é uma forma consagrada de analisar questões de múltipla escolha [Haladyna, 1999]. Ela mostra o desempenho dos quatro quartis de alunos e também resume as escolhas feitas pelos alunos em cada quartil. O eixo horizontal representa os quatro quartis de alunos. A linha de tendência superior nessa figura representa a alternativa B, a correta para a Questão 2. Como mencionado anteriormente, aproximadamente 90% dos alunos do primeiro quartil escolheram a alternativa B. A porcentagem de alunos que escolheram a alternativa B diminui no segundo quartil, mas ela continua sendo, de longe, a escolha mais popular entre os alunos desse quartil. Para os alunos do terceiro quartil, pouco mais da metade escolheu a alternativa B, mas aproximadamente 30% escolheram a alternativa A. Nessas figuras, parece que a maioria dos alunos do terceiro quartil compreendeu a função básica do código na Questão 2, embora 30% não tenham percebido o detalhe não idiomático de que os elementos na posição zero dos arrays não eram contados. Entre os alunos do quarto quartil, a opção correta foi menos popular do que a opção A, e muitos alunos escolheram as opções C ou D.

A questão 2 não é particularmente difícil. Os alunos do primeiro quartil se saíram muito bem nessa questão, e mesmo os do segundo e terceiro quartis demonstraram uma forte preferência pela resposta correta. No entanto, é uma questão eficaz para distinguir a maioria dos alunos daqueles com desempenho particularmente fraco (ou seja, do quarto quartil).

Com base nessa questão, parece que a maioria dos alunos nos três quartis superiores possui um domínio razoável dos conceitos avaliados (principalmente arrays e iteração).

Um dos aspectos mais desafiadores na elaboração de uma prova de múltipla escolha é a criação das alternativas incorretas (ou seja, as opções erradas). Uma boa alternativa incorreta deve ser plausível o suficiente para atrair alguns alunos. No entanto, ela deve ser claramente incorreta para não induzir ao erro os alunos que realmente dominam o conteúdo. Na prática, é difícil elaborar três boas alternativas incorretas. A Figura 3.3 mostra que a alternativa A foi a mais eficaz, enquanto a alternativa D não foi eficaz para os três quartis superiores.

Resumindo a discussão acima, a Figura 3.3 ilustra duas propriedades visuais geralmente consideradas desejáveis na maioria dos gráficos de questões de múltipla escolha [Haladyna, 1999], particularmente quando os alunos são avaliados de acordo com uma norma (ou seja, classificados segundo uma distribuição de notas desejada). Essas duas propriedades são:

- 1) A linha de tendência para a resposta correta diminui monotonicamente da esquerda para a direita (ou seja, dos alunos mais fortes para os mais fracos).
- 2) A linha de tendência para cada elemento distrator aumenta monotonicamente da esquerda para a direita.

No entanto, é menos claro que as questões de múltipla escolha (QMC) devam exibir consistentemente essas propriedades quando os alunos são avaliados por critérios (ou seja, quando seu domínio de determinados conhecimentos e habilidades é testado), que é o objetivo deste projeto de pesquisa. A próxima subseção discute uma QMC mais difícil do conjunto de 12, que não exibe todas as propriedades de referência à norma "desejáveis" mencionadas acima.

Pergunta 2.

Considere o seguinte fragmento de código.

```
int[] x1 = {1, 2, 4, 7}; int[] x2 = {1, 2, 5, 7}; int
i1 = x1.length-1; int i2 = x2.length-1; int count
= 0;

enquanto ((i1 > 0) && (i2 > 0)) {

    se ( x1[i1] == x2[i2] ) {

        ++contagem; --
        i1; --i2;

    } senão se (x1[i1] < x2[i2]) {

        --i2;

    } outro
    {
        // x1[i1] > x2[i2] --i1;
    }

}
```

Após o término do loop while acima, qual valor será o valor de "count"?

a) 3 b) 2 c) 1 d)

0

Figura 3.2. Questão 2 do conjunto de 12 questões de múltipla escolha.

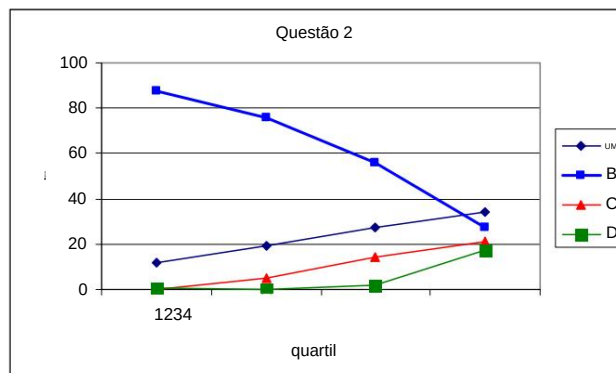


Figura 3.3. Respostas dos alunos à Pergunta 2, por quartil.

Pergunta 8.

Se dois números quaisquer em uma matriz de inteiros, não necessariamente consecutivos, estiverem fora de ordem (ou seja, o primeiro número na matriz é maior que o segundo), isso é chamado de inversão. Por exemplo, considere uma matriz "x" que contém os seguintes seis números:

4 5 6 2 1 3

Existem 10 inversões nesse arranjo, como:

$x[0]=4 > x[3]=2$ $x[0]=4 > x[4]=1$
 $x[0]=4 > x[5]=3$ $x[1]=5 > x[3]=2$
 $x[1]=5 > x[4]=1$ $x[1]=5 > x[5]=3$
 $x[2]=6 > x[3]=2$ $x[2]=6 > x[4]=1$
 $x[2]=6 > x[5]=3$ $x[3]=2 > x[4]=1$

O código-fonte abaixo tem como objetivo contar o número de inversões em um array "x":

```
int inversionCount = 0;

para (int i=0; i<x.length-1; i++) {
    para xxxxxx {
        se ( x[i] > x[j] ) ++contagem de
            inversões;
    }
}
```

Ao término da execução do código acima, a variável "inversionCount" deverá conter o número de inversões no array "x". Portanto, o "xxxxxx" no código acima deve ser substituído por:

- a) (int j=0; j<x.length; j++)
 b) (int j = 0; j < x.length - 1; j++) c) (int j = i + 1; j < x.length; j++)
 d) (int j = i + 1; j < x.length - 1; j++)

Figura 3.4. Questão 8 do conjunto de 12 questões de múltipla escolha.

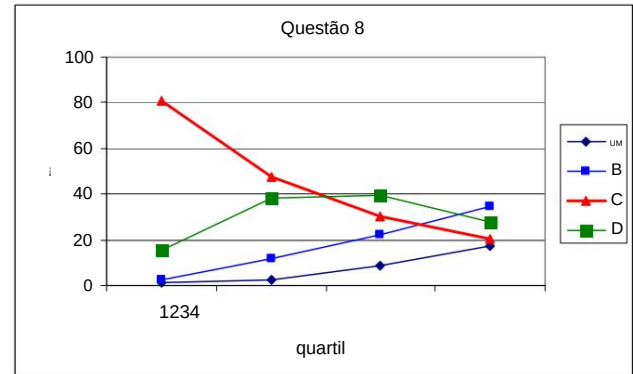


Figura 3.5. Respostas dos alunos à questão 8, por quartis.

3.2 Questão 8: Uma questão de múltipla escolha mais difícil.

A Figura 3.4 contém a Questão 8 do conjunto completo de 12 questões de múltipla escolha. Ela foi respondida corretamente por 51% de todos os alunos que a tentaram, a terceira menor porcentagem entre todas as 12 questões. As quatro alternativas apresentadas aos alunos focam em dois pontos do circuito interno:

- 1) O loop interno deve começar o processamento na posição 0 ou na posição i+1?
- 2) O loop interno deve continuar processando enquanto j < x.length ou enquanto j < x.length - 1?

As quatro combinações possíveis das duas opções acima são apresentadas como alternativas, respectivamente:

- a) Esta opção está incorreta no primeiro dos dois pontos acima, mas é a mais idiomática das quatro opções.
 Ou seja, trata-se de um loop onde a variável de controle "j" percorre toda a matriz, o que provavelmente é o primeiro e mais comum processo iterativo em uma matriz visto por esses programadores iniciantes.
- b) Esta opção está incorreta em ambos os pontos acima, mas é a opção mais semelhante ao laço externo.
- c) A resposta correta.
- d) Esta opção está incorreta no segundo dos dois pontos acima, mas possui uma condição de teste semelhante à condição de teste do loop externo.

A análise por quartis da Questão 8 é apresentada na Figura 3.5. Aproximadamente 80% dos alunos do primeiro quartil escolheram a resposta correta, a opção C. Essa opção foi a mais popular entre os alunos do segundo quartil, mas a alternativa D foi quase tão popular, com a condição de término do loop incorreta $j < x.length - 1$. A alternativa D foi, na verdade, a escolha mais popular entre os alunos do terceiro quartil. A alternativa D foi a segunda opção mais popular entre os alunos do quarto quartil; tanto ela quanto a alternativa mais popular contêm a condição de término do loop incorreta $j < x.length - 1$. Em resumo, a mais popular

A opção de distração em todos os quartis continha a condição de término do loop incorreta.

Uma análise das transcrições indica que os alunos que consideraram a resposta correta como `j<x.length-1` entenderam que o índice “j” precisava ir até o final do array e reconheceram que a última posição do array era `x.length-1`, mas negligenciaram o efeito do símbolo “<”, selecionando sua resposta como se esse símbolo fosse “<=”. Um grande número de alunos cometeu esse erro, o que é surpreendente, visto que a questão seguinte no conjunto de 12 questões de múltipla escolha exige que os alunos selecionem outra condição de teste do loop, e os alunos tiveram um desempenho muito melhor nessa questão, com 73% de acertos. É possível que os alunos tenham sido induzidos a selecionar a condição de teste incorreta na Questão 8 devido à presença, nessa mesma questão, da condição de teste “`i<x.length-1`” para o loop externo. Em uma transcrição de entrevista, um aluno do segundo quartil reconheceu explicitamente essa como a razão para a escolha: “Sim, eu escolhi essa porque seria a mesma coisa...”. Se essa conjectura estiver correta – de que os alunos se distraíram com a presença de outra condição de loop na mesma página – então isso destaca a fragilidade do conhecimento de muitos programadores iniciantes.

3.3 Desempenho Geral nas Questões de Múltipla

Escolha A Tabela 3.3 mostra a porcentagem de alunos que responderam corretamente a cada uma das 12 questões de múltipla escolha. A coluna “Classificação” mostra a dificuldade relativa de cada questão, definida pela porcentagem de alunos que responderam corretamente à questão. As questões 2 e 8, que foram estudadas em detalhes anteriormente, estão classificadas como a 6ª e a 3ª mais difíceis, respectivamente.

(Em algumas instituições, alguns alunos não responderam a todas as 12 questões de múltipla escolha. Em geral, esses alunos não podem ser incluídos na análise de dados deste artigo, pois grande parte da análise depende da classificação de cada aluno em um quartil, com base em seu desempenho em todas as 12 questões. No entanto, na Tabela 3.3, esses alunos foram incluídos.)

A questão 6 foi classificada como a segunda mais difícil. Uma análise detalhada dessa questão encontra-se no Apêndice B. Essa questão envolve o uso de um “return” dentro de um laço “for”. Pelas transcrições, fica evidente que muitos alunos não entendem que o “return” encerra o laço imediatamente. Essa concepção errônea é comum em diversas instituições e países.

Ignorando a questão 6, por envolver uma incompreensão conceitual por parte dos alunos, as questões consideradas mais difíceis na Tabela 3.3 são as questões 8, 11 e 12. Uma característica comum a essas três questões é que todas são “questões de código esqueleto”. Ou seja, essas questões exigem que o aluno selecione o código correto para concluir a atividade.

É fornecido um código “esqueleto”. Como regra geral, as questões de múltipla escolha mais fáceis são as “questões de código fixo”. Ou seja, as questões mais fáceis exigem que o aluno execute manualmente algum código e selecione o resultado. A questão 2 é um exemplo de questão de código fixo. A exceção a essa regra geral é a questão 9, que é uma questão de código esqueleto, mas que está classificada como a segunda mais fácil na Tabela 3.3.

Questão de múltipla escolha com porcentagem de acertos	t	Classificação	Não. Alunos
1	68	8/9	644
2	65	6	644
3	67	7	611
4	62	5	611
5	74 (mais fácil)	12	611
6	42	2	611
7	72	10	798
8	51	3	798
9	73	11	798
10	68	8/9	644
11	59	4	611
12	38 (mais difícil)	1	611

Tabela 3.3. Porcentagem de alunos que responderam corretamente a cada uma das 12 questões de múltipla escolha.

Por fim, observamos que é possível que a questão 12 tenha se mostrado difícil simplesmente por ser a última questão. Os alunos podem ter estado cansados nessa altura. Os alunos para quem este teste foi um fator determinante significativo da sua nota final podem ter tido a motivação necessária para superar esse cansaço. No entanto, os alunos para quem o teste teve pouca ou nenhuma importância teriam pouca motivação para superar o cansaço.

3.4 Desempenho Relativo entre Instituições A premissa fundamental de qualquer estudo multi-institucional é que os dados coletados em diferentes instituições podem ser comparados de forma significativa. Inevitavelmente, porém, haverá diferenças entre as instituições, tanto em sua natureza quanto nos detalhes da coleta de dados. Neste estudo, as diferenças incluíram:

- 1) Habilidade do aluno. Claramente, algumas instituições atraem alunos com maior talento inato para programação.
- 2) Experiência do aluno. A maioria dos alunos que forneceram dados de desempenho estava no final ou perto do final do primeiro semestre de programação, mas a população total de

Os alunos estavam matriculados desde o meio do primeiro semestre até o final do terceiro semestre.

- 3) Motivação dos alunos: Em algumas instituições, este teste teve impacto nas notas das disciplinas, em graus variados; em outras, os alunos participantes eram voluntários.
- 4) Linguagem de programação: Em uma instituição, o código Java presente nas questões de múltipla escolha foi traduzido para C++.
- 5) Modalidade do teste: Em um local, o teste foi realizado em computador; nos demais, foi realizado em papel.
- 6) Formatação da prova: Alguns pesquisadores reformatarem as questões para corresponder ao estilo de recuo utilizado em suas aulas. Além disso, um pesquisador preparou múltiplas versões das questões, com as opções de cada questão de múltipla escolha reordenadas, para evitar a cópia.

A Figura 3.6 mostra a variação no desempenho dos alunos entre as instituições, mas também revela algumas tendências gerais comuns a elas. Esta figura detalha as informações fornecidas na Tabela 3.3 por instituição, mostrando a porcentagem de alunos que responderam corretamente a cada questão. No entanto, a Figura 3.6 apresenta apenas um subconjunto dos dados da Tabela 3.3, referente às 6 instituições que forneceram dados de desempenho para pelo menos 20 alunos, todos respondendo às 12 questões de múltipla escolha. A Figura 3.6 destaca que, em qualquer questão, a porcentagem de acertos variou consideravelmente entre as instituições. Contudo, algumas tendências claras são comuns à maioria das instituições. Por exemplo, as questões 6, 8 e 12 se destacam por serem as mais difíceis na maioria das instituições.

Uma das instituições na Figura 3.6 apresenta um desempenho contrário à tendência geral da maioria das outras instituições mostradas na figura. Trata-se da instituição com uma porcentagem particularmente baixa na questão 4. Os alunos dessa mesma instituição tiveram um desempenho relativamente bom nas questões mais difíceis, 8, 11 e 12. Há três possíveis fatores que contribuem para o desempenho atípico dessa instituição. Primeiro, essa instituição forneceu dados de apenas 20 alunos, o mínimo necessário para inclusão na figura. Portanto, os dados dessa instituição são mais suscetíveis a variações devido ao pequeno tamanho da amostra. Segundo, esses alunos estavam em seu primeiro semestre de programação Java e aprenderam sobre "objetos" em Java desde cedo, o que pode não tê-los preparado adequadamente para questões sobre processos iterativos em arrays. Terceiro, a análise dos históricos escolares dessa instituição revela que esses alunos haviam estudado, ou estavam estudando em paralelo, algoritmos em um contexto independente de linguagem, e reconheceram algumas das últimas questões de múltipla escolha como sendo muito semelhantes aos algoritmos que haviam estudado. Se ignorarmos essa instituição na Figura 3.6, as tendências se tornam ainda mais evidentes entre as instituições.

Conforme explicado em uma seção anterior, uma instituição contribuiu com mais da metade dos dados de desempenho para

Os alunos responderam a todas as 12 questões de múltipla escolha. Na Figura 3.6, essa instituição é representada por losangos e segue as mesmas tendências da maioria das outras instituições.

Com base na análise desta subseção, conforme ilustrado na Figura 3.6, concluímos que existem tendências nos dados entre as instituições. Em geral, uma questão que é substancialmente mais difícil para os alunos de uma instituição também é mais difícil para os alunos de outras instituições.

3.5 Desempenho por Quartis A Tabela 3.4 mostra a porcentagem de alunos com a resposta correta para cada questão, dividida por quartil de desempenho. A coluna mais à direita dessa tabela mostra a porcentagem média de desempenho em todas as questões para cada quartil. Nas outras colunas, os números em negrito e sublinhados representam as porcentagens abaixo da média para aquele quartil. Assim como na análise de dados anterior, as questões 6, 8 e 12 se destacam por serem particularmente difíceis; aqui vemos que elas são relativamente difíceis para alunos em todos os quartis de desempenho.

A questão 11 é interessante, pois os dois quartis intermediários tiveram um desempenho abaixo de suas respectivas médias nessa questão. No entanto, a diferença de desempenho nessa questão específica em relação à média geral não ultrapassa 4% para todos os quatro quartis, portanto, o desempenho relativamente fraco dos dois quartis intermediários deve ser interpretado com cautela.

3.6 Familiaridade com provas de múltipla escolha

A maioria das instituições não avalia alunos de programação por meio de questões de múltipla escolha. As práticas de avaliação mais comuns consistem em pedir aos alunos que escrevam código ou expliquem em palavras o que um trecho de código faz. Surge, portanto, a questão de se os alunos que participaram deste estudo estavam bem preparados para a tarefa exigida.

As questões de múltipla escolha (MCQs) utilizadas neste estudo foram fornecidas por uma instituição participante, onde os alunos, ao final do primeiro semestre de programação, são aprovados ou reprovados na disciplina com base em uma prova de múltipla escolha. No restante desta subseção, nos referiremos a essa instituição como a instituição "base" e a todas as outras instituições como instituições "satélite". Os alunos da instituição base recebem rotineiramente um conjunto de MCQs para prática várias semanas antes da prova. Portanto, esses alunos estão bem preparados (e altamente motivados) para obter um bom desempenho no tipo de MCQs utilizado neste estudo. O desempenho dos alunos nas 12 MCQs da instituição base está na Tabela 3.5. O restante desta seção compara os dados dessa tabela com os dados das instituições satélite.

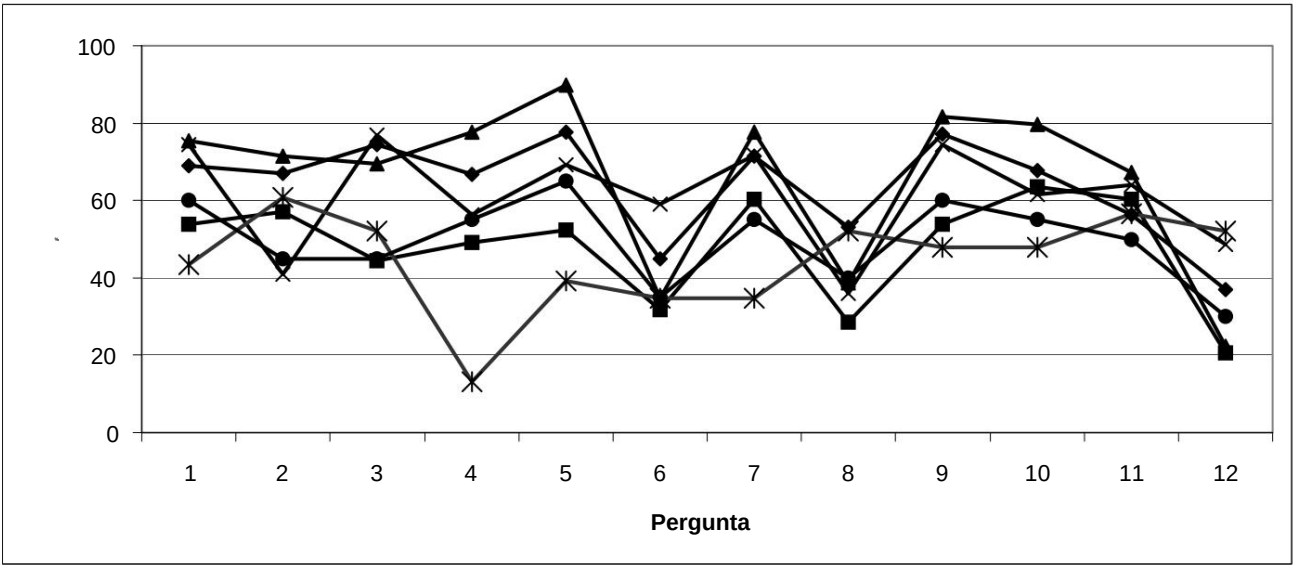


Figura 3.6. Percentagem de alunos com a resposta correta para cada questão, nas 6 instituições que forneceram dados de desempenho para pelo menos 20 alunos, nos quais esses alunos responderam a todas as 12 questões de múltipla escolha. Cada linha de tendência corresponde a uma instituição.

quartil da questão \	1º trimestre	2º trimestre	3º trimestre	4º trimestre	5º trimestre	6º trimestre	7º trimestre	8º trimestre	9º trimestre	10º trimestre	11º trimestre	12º trimestre	Média, todos os trimestres
Top (10-12)	93	<u>87</u>	99	90	99	<u>74</u>	95	<u>81</u>	98	93	89	<u>73</u>	89
Segundo (8-9)	79	76	81	73	93	<u>50</u>	82	<u>47</u>	86	81	<u>67</u>	<u>31</u>	70
Terceiro (5-7)	50	56	62	52	72	<u>28</u>	61	<u>30</u>	67	65	<u>46</u>	<u>18</u>	50
Fundo (0-4)	35	28	25	25	<u>21</u>	<u>14</u>	31	<u>20</u>	32	<u>21</u>	27	<u>13</u>	24
Todos os quartis	65	63	68	61	73	<u>43</u>	69	<u>46</u>	72	67	<u>58</u>	<u>35</u>	60

Tabela 3.4. Percentagem de alunos com resposta correta para cada questão, por quartil de desempenho.

Questão nº	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12
Percentagem	<u>84</u>	<u>79</u>	55	69	80	42	78	<u>63</u>	76	80	69	<u>67</u>

Tabela 3.5. Percentagem de alunos com resposta correta para cada questão, na instituição “base” onde os testes de múltipla escolha foram elaborados. As percentagens em negrito e sublinhadas estão fora do intervalo de percentagens das outras instituições na Figura 3.6.

Na Tabela 3.5, as quatro percentagens em negrito e sublinhadas estão fora do intervalo de percentagens das instituições satélite na Figura 3.6. Em todos os quatro casos, a percentagem de alunos da instituição base na tabela é maior do que a percentagem de alunos das instituições satélite na figura.

As questões 1 e 2 foram respondidas de forma mais insatisfatória pelos alunos do campus satélite do que pelos alunos do campus base, o que pode indicar que os alunos do campus satélite levaram algumas questões para...

familiarizar-se com o que era exigido deles neste teste.

Uma diferença notável entre as Tabelas 3.3 e 3.5 reside na percentagem de alunos que responderam corretamente à Questão 12. Na instituição de origem, 67% dos alunos responderam corretamente a essa questão, em comparação com apenas 38% nas instituições satélite. Na instituição de origem, uma pequena variação na Questão 12 fazia parte de um conjunto de

Questões práticas foram fornecidas aos alunos. Portanto, espera-se que os alunos da instituição de origem tenham um desempenho melhor nessa questão.

Concluímos que a falta de familiaridade com exames de múltipla escolha não foi um grande obstáculo para os alunos de instituições satélite. Suspeitamos que, se os alunos das instituições satélite também tivessem tido acesso a um extenso conjunto de questões práticas antes de tentarem responder a essas 12 questões de múltipla escolha, seu desempenho poderia ter sido um pouco melhor, mas não substancialmente melhor.

(Encerramos esta subseção com a seguinte nota de esclarecimento. A instituição de origem forneceu dados para todas as 12 questões de múltipla escolha de apenas três alunos. Com exceção da Tabela 3.3, a análise de dados neste artigo diz respeito aos alunos que responderam a todas as 12 questões de múltipla escolha. Portanto, os dados da instituição de origem não podem introduzir um viés significativo na análise de dados.)

3.7 Confiabilidade

Existem métodos estabelecidos na literatura educacional para avaliar a "confiabilidade" de um teste.

A confiabilidade tem um significado bastante restrito aqui: um teste é confiável se discriminar consistentemente entre os alunos, com base na divisão das questões em grupos, na correção dos grupos e na verificação da concordância dos resultados. Portanto, os testes de confiabilidade baseiam-se na premissa de que todas as questões em uma prova de múltipla escolha avaliam os mesmos conhecimentos e habilidades. Calculamos o coeficiente alfa de Cronbach (Ebel e Frisbie, 1986), que fornece a correlação média entre as pontuações de todas as possíveis divisões das questões em dois grupos. Usando os dados dos alunos que responderam a todas as 12 questões de múltipla escolha (N=556), obtivemos um valor de 0,75. Por convenção, um valor de 0,80 ou superior é considerado "confiável". No conjunto de 12 questões de múltipla escolha usado neste estudo, as questões 6, 8 e 12 (um quarto do conjunto completo de questões) destacam-se por serem relativamente difíceis. Portanto, não é surpreendente que a confiabilidade do conjunto completo de 12 questões de múltipla escolha fique um pouco abaixo do limite convencional de 0,80.

3.8 Tempo gasto para responder às doze questões de múltipla escolha

É possível que alguns alunos tenham tido um desempenho ruim nas questões de múltipla escolha mais para o final do conjunto de 12 por falta de tempo. O tempo a ser concedido aos alunos para concluir a prova não foi especificado como parte do projeto do experimento, além da recomendação às instituições participantes de que os alunos deveriam ter pelo menos uma hora para responder a todas as 12 questões de múltipla escolha.

Os dados de duração foram coletados para 339 dos alunos que realizaram os 12 testes de múltipla escolha. A maior parte desses dados de tempo veio da instituição que contribuiu com a maioria dos dados de desempenho, onde não havia limite máximo de tempo estabelecido.

Metade dos alunos levou entre 30 e 60 minutos para responder às 12 questões. Ou seja, 75% de todos os alunos concluíram as 12 questões de múltipla escolha em menos de uma hora. O maior tempo gasto por um aluno no primeiro quartil de desempenho (ou seja, com pontuação de 10 a 12) foi de 105 minutos. O maior tempo gasto por qualquer aluno foi de 115 minutos, por um aluno no terceiro quartil de desempenho (ou seja, com pontuação de 5 a 7). A distribuição dos tempos para concluir as 12 questões de múltipla escolha foi semelhante nos três quartis de desempenho superiores.

Os alunos no quartil inferior tenderam a ser mais rápidos. Metade dos alunos do quartil inferior levou entre 20 e 50 minutos para responder às 12 perguntas, sendo que o que levou mais tempo foi de 65 minutos.

3.9 Discussão dos Dados de Desempenho Os

alunos que se encontram no quartil inferior de desempenho, com pontuação de 0 a 4 em 12, provavelmente sofrem de problemas mais fundamentais do que uma incapacidade de resolver problemas. Alunos que se encontram no quartil superior provavelmente possuem um sólido domínio dos fundamentos da programação. Se alunos do quartil superior demonstrarem posteriormente dificuldades na escrita de código, então provavelmente apresentam uma dificuldade na resolução de problemas.

Após categorizar os quartis superior e inferior, resta a tarefa mais árdua de observar os 50% dos alunos que se encontram no meio, aqueles que obtiveram notas entre 5 e 9. É interessante considerar o seguinte argumento hipotético. Suponha que os alunos que participaram deste estudo estivessem cursando o primeiro semestre de programação em uma única instituição. Suponha ainda que eles tivessem recebido essas 12 questões de múltipla escolha como prova, e que a instituição considerasse uma taxa de reprovação de 25% como o limite máximo aceitável. Nesse caso, os alunos que obtivessem nota 5 de 12 nessas questões seriam aprovados para o segundo semestre do curso de programação. Na visão dos autores, alunos com nota 5 de 12 nessas questões provavelmente não possuem o nível de habilidade e conhecimento necessário para um curso subsequente. Os autores deixam ao leitor a tarefa de definir sua própria nota de aprovação aceitável nessas 12 questões e calcular a taxa de reprovação resultante para essa turma hipotética.

Uma observação que fazemos sobre os 50% dos alunos com desempenho mediano é que, pelo fato de terem respondido corretamente a algumas perguntas, demonstraram uma compreensão conceitual de loops e arrays. Portanto, a dificuldade desses alunos não reside na incompreensão das estruturas da linguagem. Sua dificuldade está na incapacidade de percorrer de forma confiável a longa cadeia de raciocínio necessária para executar o código manualmente e/ou na incapacidade de raciocinar de forma confiável em um nível mais abstrato para selecionar a linha de código faltante. Com base nos dados de desempenho, não é possível chegar a conclusões definitivas sobre a causa exata dessa dificuldade. Para explorar essa questão,

Além disso, o restante do artigo examina os outros dados coletados neste projeto, os dados dos rabiscos e as transcrições das entrevistas.

4. RABISCOS Ao se

depararem com um trecho de código para ler e entender, programadores experientes frequentemente “rabiscam”. Ou seja, desenhavam diagramas e faziam outras anotações como parte do processo de compreensão da função do código.

A Figura 4.1 mostra as anotações feitas por um aluno ao escolher corretamente a opção “c” (indicada pelo círculo ao redor dessa opção) para a Questão 1. (Lembre-se de que, como mostrado na Tabela 3.3, essa questão foi respondida corretamente por 68% de todos os alunos, classificando-a como uma das questões de múltipla escolha mais fáceis utilizadas neste estudo.) Acima da primeira linha do código Java na Figura 4.1, onde o array “x” é inicializado, o aluno anotou as diversas posições do array. Neste estudo, nos referimos a isso como um rabisco de “posição” (ou simplesmente “P”). Acima da condição de teste do laço “while”, o aluno escreveu os valores das variáveis.

Nessa condição de teste, à medida que as variáveis “sum” e “i” mudavam, os valores atualizados eram escritos ao lado do valor antigo. Chamamos isso de rabisco de “número” (“N”).

Acima e à direita da condição de teste, o aluno escreveu os valores variáveis em diversas expressões booleanas, como “0 < 3” e “2 < 3”. Este é um exemplo de um rabisco de “rastreamento” (“T”). Abaixo da instrução “sum += x[i];”, o aluno escreveu um rabisco de “cálculo” (“C”), “sum = 1 + 4 = 5”. Próximo às quatro opções, o aluno escreveu “sum = 0” e “i = 0” para registrar o valor dessas duas variáveis; esses são outros exemplos de rabiscos “N”. No canto inferior direito, há uma tabela com uma linha de cabeçalho contendo “sum”, “lim”, “i” e “len”. Conforme o aluno executava o código manualmente e as variáveis listadas na linha de cabeçalho dessa tabela mudavam de valor, os novos valores eram inseridos na tabela. Este é um exemplo do tipo de rabisco mais elaborado identificado neste estudo, o “rastreamento sincronizado” (“S”).

4.1 Categorização dos Rabiscos

Neste estudo, analisamos os rabiscos de 56 estudantes, sendo os 37 entrevistados e outros escolhidos aleatoriamente entre todas as instituições participantes. Dois autores deste estudo analisaram conjuntamente as páginas coletadas desses estudantes e desenvolveram um conjunto preliminar de categorias para os rabiscos.

Após os dois autores terem elaborado as categorias preliminares, três outros autores analisaram separadamente um subconjunto das mesmas páginas examinadas pelos dois primeiros autores e contabilizaram a frequência de ocorrência de cada tipo de rabisco, conforme definido na categorização preliminar. As frequências de cada um desses três autores foram comparadas.

em relação às frequências obtidas pelos dois primeiros autores. Nos três casos, a discrepância foi de aproximadamente 10%.

Algumas das discrepâncias foram causadas simplesmente por um autor ter deixado passar um desenho, enquanto outras ocorreram porque as explicações das categorias não eram suficientemente claras. As explicações foram reescritas para esclarecer as categorias e produzir a versão final das categorias de rabiscos.

A Tabela 4.1 contém a versão final das categorias de rabiscos, apresentando todos os tipos de rabiscos identificados neste estudo. Observe que qualquer marca em uma prova é considerada um rabisco, exceto a indicação da opção de múltipla escolha selecionada. Observe também que uma única questão de múltipla escolha respondida por um aluno pode ter várias categorias de rabiscos atribuídas a ela, como seria o caso da questão mostrada na Figura 4.1.

4.2 Categoria de rabiscos e precisão das respostas

Uma questão óbvia a ser considerada é a eficácia de cada uma dessas categorias de rabiscos para responder às questões de múltipla escolha. Para investigar essa questão, analisamos os rabiscos dos 56 alunos em todas as 12 questões de múltipla escolha, da seguinte forma. Para cada categoria de rabisco, calculamos a porcentagem de questões respondidas corretamente usando aquele rabisco (como uma porcentagem de todas as questões em que aquela categoria de rabisco apareceu). Essas porcentagens são apresentadas na Tabela 4.2. A coluna “Pontos de Dados” nessa tabela indica o número total de ocorrências de cada tipo de rabisco. O número máximo possível de pontos de dados para cada tipo de rabisco é 56 alunos × 12 questões de múltipla escolha = 672.

Não surpreendentemente, a Tabela 4.2 indica que, se um aluno traçar cuidadosamente o código (S, T ou O), documentando assim as mudanças nas variáveis, a probabilidade de obter a resposta correta é alta. Em contrapartida, não fazer rabiscos (B) leva à resposta correta em apenas 50% das vezes. Embora essas estatísticas não surpreendam muitos professores, são dados muito úteis para que eles os apresentem aos seus alunos.

Note que um aluno pode usar mais de uma categoria de desenho para responder a uma pergunta. Portanto, algumas categorias, que têm pouca relação com a determinação da resposta correta, podem aparecer em conjunto com outra categoria de desenho que é a principal responsável por responder à pergunta (por exemplo, o desenho de posição sozinho provavelmente não levará a uma resposta correta). Isso também pode explicar o sucesso de 100% do desenho de contagem (K). De qualquer forma, o número de pontos de dados para o desenho de contagem é muito baixo para ser considerado significativo.

1. Consider the following code fragment:

```

    int x[] = {2, 1, 4, 5, 7};
    int length = 5;
    int limit = 3;
    int i = 0;
    int sum = 0;

    while ( (sum < limit) && (i < length) )
    {
        ++i;
        sum += x[i];
    }

```

What value is in the variable "i" after this code is executed?

- a) 0
- b) 1
- c) 2
- d) 3

sum = 0 i = 0

	sum	lim	i	len
1	0	3	0	5
2		3	1	5
3		3		5
4		3		5

sum =

sum = 1

sum = 4 + 1 = 5

Figura 4.1 Rabiscos de um aluno para a Questão 1

Código	Nome	Descrição	Exemplos
	Resposta alternativa: O aluno alterou sua resposta para esta pergunta.		 d.
	Página em branco B	Sem rabiscos para esta pergunta.	
	Computação em C	Um cálculo aritmético ou booleano. Reescrever uma comparação não foi contabilizado como um cálculo (ver N).	$3 + 5$ $3 == 5$ falso
	E. Marcas Extrínsecas: Marcas que parecem sem significado ou são ambíguas (ou seja, não puderam ser caracterizadas definitivamente pelo pesquisador). Inclui itens como: elementos de matriz circulados; setas sem significado; pontos na página; números riscados diversos, etc. Não inclui uma cópia riscada de um traçado (uma repetição óbvia do traçado).		
	K Mantendo a Contagem	Algum valor está sendo contabilizado várias vezes (variável específica não indicada).	
	Número N	Mostra o valor de uma variável. Geralmente usado em comparações. Diferente de S ou T abaixo. Os valores caracterizados como números foram associados a variáveis cujos valores não mudaram no fragmento de código.	5 0 soma<limite - ou - x.comprimento 6
	O Traço Ímpar	Um tipo estranho de traço (ou seja, usava setas, não pôde ser caracterizado como S ou T abaixo), mas parecia ser um traço. De acordo com o exemplo dado aqui, esses rabiscos podem ser representações pictóricas de matrizes, "antes" e "depois" das operações na matriz.	0 1 2 // 1 2
P	Posição	Imagem da correspondência entre a posição (índice) do elemento da matriz e o valor do elemento. Observação: no exemplo, a linha superior contém os índices e a linha inferior exibe a matriz impressa,	0 1 2 3 x={2 4 6 8}
S	Sincronizado Rastrear	conforme mostrado na prova. Mostra os valores de múltiplos elementos sempre que um deles muda. Essencialmente, é uma tabela. Em alguns casos, os alunos também dedicam colunas da tabela a "variáveis" que não se alteram durante a execução do código.	<u>eu soma</u> 0 0 0 3 1 3
	Traço T	Mostra os valores de uma variável (não necessariamente em formato de tabela) à medida que ela muda (ou seja, mostra mais de um valor para pelo menos uma variável). - ou - O valor de uma variável foi sobrescrito com um novo valor.	i1 = 1 2 3 4 - ou - i1 = 1 i1 = 2 - ou - i1 0 1
	U Sublinhado	Parte da pergunta foi sublinhada ou sombreada pelo aluno para dar ênfase.	
	X descartado	Uma ou mais respostas alternativas foram riscadas, de modo que a resposta parecesse ter sido selecionada por eliminação.	

Tabela 4.1. Categorização de rabiscos

Categoria de rabiscos	%Correto	Pontos de dados
Contagem (K)	100	6
Traço Ímpar (O)	78	23
Rastreamento Sincronizado (S)	77	73
Traço (T)	75	215
Resposta alternativa (A)	69	26
Posição (P)	64	75
Número (N)	70	189
Computação (C)	60	30
X descartado (X)	60	60
Marcas estranhas (E)	57	89
Sublinhado (U)	52	44
Página em branco (B)	50	256

Tabela 4.2: Percentagem de respostas corretas quando os alunos (N=56) usam um determinado tipo de desenho.

4.3 Rabiscos em Páginas em Branco

Uma categoria na Tabela 4.1 é o rabisco em "página em branco" ("B"), que, na verdade, indica que não houve rabiscos para aquela questão. Dada a propensão de programadores experientes a rabiscar, seria razoável esperar que programadores novatos fizessem o mesmo, mas nem sempre é esse o caso. Por exemplo, Thomas, Ratcliffe e Thomasson (2004) descreveram suas experiências ao incentivar alunos a rabiscar. Eles relatam que, quando um aluno aborda um instrutor com um problema, o aluno "frequentemente fica impaciente quando o instrutor recorre a desenhar um diagrama, e depois se surpreende que a abordagem funcione" [p. 250]. Eles também relatam que, em uma ocasião em que testaram alunos e forneceram a cada aluno uma folha de papel na qual ele podia rabiscar livremente, quase dois terços das páginas devolvidas estavam em branco.

Neste estudo, dos 56 alunos cujos dados de rabiscos foram examinados, um quinto deles não fez nenhum rabisco (B) ao responder à Questão 2, e mais da metade (55%) não rabiscou na Questão 8. Essas taxas relativas de rabiscos para as Questões 2 e 8 parecem contra-intuitivas. Ao contrário dos novatos neste estudo, relatos sugerem que programadores experientes costumam rabiscar quando tentam entender um trecho de código difícil.

A questão 2 é mais fácil do que a questão 8, mas os alunos que estudamos estão menos inclinados a fazer rabiscos para a questão mais difícil. (Essa questão será abordada mais detalhadamente na discussão das transcrições da Pergunta 8.)

Davies (1996) relata que, ao escrever programas, os especialistas fazem uso extensivo de registros externos para gerenciar

enquanto os novatos dependem muito da sua memória de trabalho de curto prazo. A partir deste estudo com grupo de trabalho, parece que os novatos também dependem muito da sua memória de trabalho de curto prazo ao tentar decifrar o código.

A frequência real de rabiscos em branco para as questões 2 e 8 provavelmente subestima a falta geral de rabiscos eficazes. Parece improvável que qualquer aluno que tenha usado apenas o rabisco de posição, e talvez algumas das outras categorias de rabiscos, tenha obtido alguma vantagem ao responder às questões 2 e 8. Portanto, a porcentagem de alunos que não fizeram nenhum rabisco eficaz provavelmente é ainda maior para as questões 2 e 8.

Existem três explicações possíveis para um aluno responder a uma questão de múltipla escolha com um rabisco em forma de "B": 1) A

questão de múltipla escolha é relativamente simples.

- 2) O aluno internalizou uma estratégia de raciocínio sofisticada para responder a esse tipo de questão de múltipla escolha.
- 3) O aluno está chutando ou usando heurísticas para selecionar uma resposta plausível sem realmente entender a questão de múltipla escolha, o que é essencialmente um palpite fundamentado.

Dada a alta dificuldade da Questão 8, parece improvável que a primeira das explicações acima seja plausível. Uma avaliação das outras duas explicações requer uma análise mais detalhada dos dados da transcrição, o que será feito na próxima seção.

5. TRANSCRIÇÕES Após

analisar os dados de desempenho dos alunos e seus padrões de rabiscos, esta seção esclarece a análise anterior por meio de um exame das transcrições das entrevistas. Enquanto as outras seções foram predominantemente quantitativas, esta seção é predominantemente qualitativa.

5.1 Demonstrações passo a

passo Na análise das transcrições a seguir, há frequentes referências a "demonstrações passo a passo". Usamos esse termo para descrever qualquer transcrição de uma questão em que o aluno executa o código manualmente, com detalhes meticulosos. A Figura 5.1 contém uma demonstração passo a passo para a Questão 2.

Perkins et al. (1989) referem-se às revisões de código como "rastreamento minucioso". Eles observam que, embora o rastreamento minucioso seja uma habilidade importante para diagnosticar erros, ele exige muito esforço mental e, portanto, muitos alunos não rastreiam seu código com atenção. Perkins et al. também observam que os erros são frequentemente ignorados durante o rastreamento porque o aluno projeta suas próprias intenções no código.

A análise da transcrição nesta seção foi feita de forma totalmente independente da análise do rabisco. Parece plausível que

Uma descrição detalhada em uma transcrição é uma evidência de um estilo de rabisco minucioso, mas não investigamos essa conexão.

5.2 Raciocínio de Nível Superior Existe

uma vasta literatura sobre modelos mentais para programadores, onde os programas são representados em um nível superior ao do próprio código. Grande parte dessa literatura concentra-se no conceito de esquema [Soloway e Ehrlich, 1984; Rist, 1986 e 2004; Detienne, 1990]. Esquemas são uma solução abstrata para um problema de programação, que pode ser aplicada em diversas situações. Conceitos intimamente relacionados incluem "planos", "templates", "idiomas" e "padrões". [Clancy e Linn, 1999].

Grande parte da literatura sobre esquemas se concentra na escrita de programas, mas os esquemas também são usados em modelos de compreensão de programas. Em alguns modelos de leitura, a compreensão é um processo "de baixo para cima", onde elementos do texto sugerem o esquema que pode ter sido usado pelo autor do programa. Outros modelos são "de cima para baixo", onde os esquemas sugerem elementos a serem procurados no texto do programa. Outros modelos permitem uma mistura de processos "de baixo para cima" e "de cima para baixo". Os modelos de esquemas preveem corretamente que um programador experiente entenderá/memorizará mais rapidamente o código que está em conformidade com um esquema do que o código que não está em conformidade [Soloway, Adelson e Ehrlich, 1988].

Alguns defenderam que os alunos deveriam aprender esquemas explicitamente [Soloway, 1986; Clancy e Linn, 1999]. Recentemente, defendeu-se que os alunos aprendessem explicitamente a reconhecer e usar dez funções comuns de variáveis [Kuittinen e Sajaniemi, 2004].

O estudo deste grupo de trabalho foi concebido principalmente para comparar o desempenho dos alunos em diferentes instituições e países. O desenho do estudo não permite uma análise detalhada de quaisquer estratégias de compreensão de nível superior utilizadas pelos alunos. Os alunos foram simplesmente solicitados a encontrar a resposta correta para cada questão de múltipla escolha, e não a articular qualquer processo de raciocínio de nível superior. No entanto, algumas inferências cautelosas podem ser feitas a partir das transcrições. Por exemplo, se um aluno escolhe uma resposta sem uma análise completa do código, então ou ele chutou ou empregou algum tipo de estratégia de raciocínio de nível superior. O inverso, porém, não é necessariamente verdadeiro. Se um aluno realiza uma análise completa para encontrar a resposta, não se pode inferir que ele não fez nenhuma inferência de nível superior sobre o código. Por exemplo, depois que um aluno escolheu sua resposta (incorreta) para a Questão 2, por meio de uma análise completa, ocorreu a seguinte troca de palavras entre o entrevistador e o aluno:

Entrevistador: *"Quando você estava fazendo esse exercício, você lia o código linha por linha. Você chegou a formar uma ideia intuitiva do que o programa estava fazendo?"*

Aluno: *"Hum, sim, eu sabia que contaria o número de elementos iguais. Não tinha certeza, à primeira vista, do que os sinais de menor que e maior que fariam."*

Embora uma análise detalhada das estratégias de compreensão de nível superior não seja possível neste estudo, as transcrições permitem investigar uma questão importante: se os alunos com melhor desempenho utilizam uma abordagem qualitativamente diferente da dos alunos com pior desempenho. É possível que todos os alunos estejam utilizando a mesma abordagem qualitativa, mas os alunos com pior desempenho cometam mais erros.

5.3 Chutes: As

provas de múltipla escolha são frequentemente criticadas por serem passíveis de resposta por palpite. Analisamos as transcrições para verificar a existência de evidências de palpites. Duas formas de palpite foram observadas nas transcrições. Às vezes, os alunos faziam uma suposição fundamentada. Por exemplo, um aluno poderia realizar uma análise preliminar incompleta, ou algum outro processo, seguido de uma afirmação como "então deve ser A ou C – vou chutar A". Considerava-se que uma suposição havia ocorrido quando um aluno fazia uma afirmação como "estou apenas chutando" ou "estou completamente perdido, apenas escolhi uma".

Nas transcrições, encontramos evidências de que os alunos chutaram em 10% de todas as respostas às perguntas, sendo aproximadamente metade delas palpites fundamentados e a outra metade palpites puros. No entanto, as taxas de palpite por questão variam bastante. Detectamos palpites em níveis superiores a 10% em cinco questões: Questão 10 (30%), Questão 12 (16%), Questão 8 (15%), Questão 6 (13%) e Questão 9 (12%). Observamos que a maioria dessas questões foi identificada como difícil na análise de desempenho. Não detectamos nenhuma evidência de palpite na Questão 7. Como esperado, observou-se que os alunos que chutaram significativamente mais do que seus colegas tiveram um desempenho notavelmente pior.

Desta vez, vou primeiro analisar o que a questão pede. Então, estou tentando encontrar o valor de `count`. Agora, vou verificar o valor de cada variável e o que está acontecendo dentro do programa. Vou anotar que o comprimento de `x1` é 4 e o comprimento de `x2` é 4. E então, no loop, verifica-se se 4 é maior que 0 e 4 é maior que 0. Dentro do loop, verifica-se se... [indecifrável] ... verifica-se, percebendo que não subtraí 1 de cada um dos valores de `i1` e `i2`. Então, altera-se `i1` para 3 e `i2` para 3. Assim, quando o terceiro

Como o último elemento de cada array é igual, entramos no primeiro loop `if`, então `count` agora é igual a 1; `i1` é igual a 2 e `i2` é igual a 2. Agora estamos percorrendo o loop novamente. 2 é maior que 0 e 2 é maior que 0, então estamos dentro do loop. Verificando o primeiro "if": os segundos índices são iguais entre si? Não. Então vamos para o `else if` e verificamos '4 é menor que 5?', que é verdadeiro, então `i2` é subtraído de 1. `i2` agora é igual a 1 e estamos voltando para o loop `while` novamente. 2 é maior que 0 e 1 é maior que 0. Então verificamos o primeiro `if`: '4 é igual a 2?', que é falso, então verificamos o segundo `if`. A condição "4 é menor que 2?" é falsa, então executamos a instrução else, que subtrai 1 de i1. Assim, i1 agora é igual a 1. O loop while continua. 1 é maior que 0 e 1 é maior que 0. Verificamos a primeira condição if: "2 é igual a 2?", que é verdadeira, então incrementamos count em 1. Count agora é 2. Subtraímos 1 de i1, que se torna 0, e de i2, que se torna 0.

Agora o loop while falha porque 0 não é maior que 0. Então ele pede o valor de count, que é 2.

Figura 5.1 Transcrição completa da resposta correta de um aluno à Questão 2, obtida por meio de um passo a passo.

5.4 Questão 2 por Quartil. Lembre-se

de que a Questão 2 é uma questão de código fixo de dificuldade média. Nesta subseção, examinamos os históricos escolares dos alunos nessa questão por quartil de desempenho.

Das trinta e sete transcrições, vinte foram excluídas da análise da Questão 2 pelos seguintes motivos. Em quatro casos, não houve gravação ou a gravação da Questão 2 foi parcial, devido a erros no uso do equipamento. Em dois casos, as respostas dos alunos a todas as 12 questões não foram registradas no banco de dados de desempenho, portanto, os alunos não puderam ser alocados a um quartil. Em sete casos, não havia uma descrição clara na transcrição da resposta escolhida pelo aluno, ou havia uma discrepância entre a aparente escolha de resposta na transcrição e a resposta registrada no banco de dados (em pelo menos um caso, é provável que o aluno tenha verbalizado uma opção de resposta, mas marcado outra em sua folha de respostas). Em sete casos, o aluno havia realizado o teste antes da entrevista, como parte da coleta de dados de desempenho, e o

A entrevista foi um "debriefing". Ou seja, a entrevista consistiu na recordação dos alunos sobre como eles abordaram a resposta à pergunta.

5.4.1 Alunos do Quartil Superior

Havia oito históricos escolares de alunos do quartil superior. Todos os alunos responderam corretamente e todos analisaram o código meticulosamente. Todos esses alunos articularam sua análise com detalhes suficientes para que um leitor da transcrição possa acompanhar o raciocínio (como na Figura 5.1).

Ao elaborarem a resposta, nenhum desses alunos demonstrou ter percebido a intenção do código, que era contar o número de elementos idênticos nas duas matrizes (em posições acima da posição zero).

5.4.2 Alunos do segundo quartil

Havia cinco históricos escolares de alunos do segundo quartil. Três alunos responderam à pergunta corretamente. Dois desses alunos apresentaram uma explicação detalhada e minuciosa, com a mesma clareza dos alunos do quartil superior. O terceiro aluno apresentou uma explicação breve e pouco clara.

"x1 é um vetor com quatro inteiros, x2 é um vetor com quatro inteiros. i1 é igual a três e i2 é igual a três, i1 e i2 são maiores que zero, então são iguais a três e x1 é igual a quatro nesta afirmação e x2 é igual a cinco nesta afirmação. i1 é igual a 2 e i2 é igual a 2, ainda maiores que zero, então x1 será igual a dois e x2 será igual a 2... portanto a resposta será B."

Dos dois alunos que responderam incorretamente, um escolheu a opção A. Essa é a opção que um aluno escolheria se compreendesse a intenção geral do código, mas não percebesse que a comparação dos elementos nos arrays para sem examinar os primeiros elementos dos arrays. Diferentemente dos alunos do quartil superior, esse aluno fez comentários explícitos e espontâneos sobre a intenção do código.

Logo no início de suas deliberações, o aluno comentou que o código usava "nomes de variáveis sem sentido". No meio das deliberações, ele comentou: "Eu consigo ver que é isso que está acontecendo, mas estou fazendo devagar para não estragar tudo". Tendo chegado à conclusão incorreta de que a resposta era a opção A, ele comentou: "Eu já imaginava que isso ia acontecer", acrescentando que o código "basicamente conta dígitos semelhantes nos arrays". Esse aluno demonstrou um estilo de leitura que gostaríamos de incentivar, no qual o aluno abstrai do código de baixo nível para um esquema de nível superior. No entanto, por não ter verificado com atenção, ele não percebeu que a contagem para sem examinar as primeiras posições dos arrays.

O aluno restante escolheu a alternativa C. Embora o aluno parecesse entender o código passo a passo, a transcrição é breve e confusa:

“Existem duas variáveis, i1 e i2, que começam em 3. Dois arrays, x1 e x2. Um laço condicional que se repete enquanto ambos forem maiores que zero, compara x1 e x2 (os elementos nas posições i1 e i2, que são iguais a 3), verifica se são iguais (e são). Ele incrementa a variável count e decrementa i1 e i2, e o laço se repete.”

E compara os segundos elementos de x1 e x2, mas desta vez eles são diferentes e x1 é menor que x2, então desta vez decrementa i2 e o loop recomeça. Desta vez compara o segundo elemento de x1 com o terceiro elemento de x2 e eles não são iguais e novamente decrementa i2 – desta vez é 0, então falso, portanto a contagem foi incrementada apenas uma vez.”

5.4.3 Alunos do Terceiro Quartil

Havia dois históricos escolares de alunos do terceiro quartil. Ambos os alunos analisaram o código, mas escolheram incorretamente a opção A.

Um desses alunos não conseguiu acompanhar bem o passo a passo e voltou atrás duas vezes depois de perceber que havia cometido um erro. Na terceira tentativa, o aluno não conseguiu acompanhar corretamente o valor da variável “i2”: *“Então, primeiro*

eles são três, i1 e i2. Pegamos os setes, eles são iguais, a contagem se torna um. E agora eles se tornam 2, e então vem... sim, eles se tornam dois, x1 e x2. Pegamos 4 e 5, vamos para o else-if. i2 é diminuído em um. Ele se torna um. E i2 ainda é 2.”

Em seguida, pegamos o segundo valor, ou seja, 4 e 2, e vamos para a última seção. Lá, i1 é reduzido. i1 torna-se 1 e i2 torna-se 2.

A transcrição acima continua, mas o erro foi cometido.

O outro aluno demonstrou uma compreensão frágil da diferença entre uma posição em uma matriz e o conteúdo dessa posição:

“... Então eu incrementei count, que seria de 0 para 1, subtraí i1 e subtraí i2, então eles moveram os ponteiros. Por “ponteiros”, o aluno se refere às variáveis i1 e i2. Eu movi os ponteiros, então agora o ponteiro x1 estaria em 4 e o ponteiro x2 estaria em 5”. O aluno parece querer dizer que x1[i1] contém 4 e x2[i2] contém 5. No entanto, ele continua dizendo “... eles ainda são iguais, então, provavelmente, faça o primeiro passo novamente. Então count é 2”. Com “ambos ainda iguais”, o aluno está afirmando que a condição booleana x1[i1] == x2[i2] é verdadeira, o que não só está incorreto, como contradiz o que ele disse imediatamente antes: “o ponteiro x1 estaria em 4 e o ponteiro x2 estaria em 5”. Ele interpretou incorretamente a condição booleana como “i1 == i2”.

Muitos professores reconheceriam o problema deste aluno como comum entre os iniciantes, e também é reconhecido.

na literatura [du Boulay, 1989]. Para este aluno, a confusão entre a posição do arranjo e o conteúdo do arranjo é mais um caso de fragilidade do que um equívoco completo, visto que o aluno respondeu corretamente a outras cinco questões. De fato, o aluno respondeu incorretamente às quatro primeiras questões devido a essa confusão, antes de perceber seu erro, e prosseguiu respondendo corretamente a 5 das 8 questões de múltipla escolha restantes, incluindo as mais difíceis.

5.4.4 Alunos do quartil inferior

Havia dois históricos escolares de alunos do quartil inferior. Ambos os alunos admitiram ter chutado.

5.4.5 Relatos dos alunos sobre a Questão 2: Como

mencionado anteriormente, algumas transcrições não foram incluídas na análise por quartis da Questão 2, pois se tratavam da lembrança dos alunos sobre como haviam abordado a questão. No entanto, é interessante examinar esses relatos para entender como os alunos enxergam o processo. Os trechos a seguir foram extraídos dos relatos de alguns alunos do terceiro quartil, que responderam corretamente à Questão 2: 1) *“... e basicamente o que eu fiz foi tentar anotar todos os valores iniciais para não me perder*

no processo caso pensasse em muita coisa ao mesmo tempo. E, novamente, eu repetia os mesmos passos, tentando acompanhar as mudanças nas variáveis e fazendo pequenas anotações ao lado”.

2) *“É exatamente o mesmo tipo de problema da questão 2... é a mesma maneira que tento resolvê-lo... anotando as variáveis e, a cada iteração, tentando descobrir quais são as variáveis... é apenas uma forma de trabalhar semelhante a usar System.out.println para descobrir os valores das variáveis. Imagino que esse seja o método usado manualmente. Isso se repete em muitas dessas questões: trata-se de memorizar alguns números, seguir a estrutura do programa e ver o que acontece. E tentar lembrar os números... todas as questões são mais ou menos como a questão 1”.*

3) *“O desafio é manter uma imagem mental, continuar observando... o estado das variáveis, dependendo de como as comparações se desenrolam... no geral, acho-as bastante difíceis...”*

Passo algum tempo pensando sobre o que acontece na primeira execução do loop, o que acontece na segunda, e fico de olho nos contadores individuais... ”.

5.4.6 Discussão das transcrições da Questão 2

Devido ao pequeno número de transcrições que puderam ser analisadas, não é possível tirar conclusões definitivas. No entanto, algumas observações provisórias interessantes podem ser feitas e poderiam ser confirmadas com a coleta de mais transcrições.

A maioria dos alunos nos três quartis superiores respondeu à pergunta por meio de uma análise passo a passo. A principal diferença entre os alunos do quartil superior e os dos outros dois quartis é que os alunos do quartil superior foram mais metódicos no processo de análise passo a passo e, portanto, cometeram menos erros.

Poucos alunos em qualquer quartil raciocinaram explicitamente em um nível superior ao da explicação passo a passo. Ou seja, poucos alunos articularam a intenção do código, que era contar o número de elementos comuns em partes dos dois arrays.

5.5 Questão 8 por Quartil Lembre-se de

que a Questão 8 é uma questão de múltipla escolha com código-esqueleto e uma das questões mais difíceis do conjunto de 12. Nesta subseção, examinamos os históricos dos alunos nessa questão por quartil de desempenho.

Das trinta e sete transcrições, vinte e quatro foram excluídas da análise da Questão 8, pelas mesmas razões que as transcrições foram excluídas da Questão 2.

Na análise anterior dos dados de desempenho, foi demonstrado que a opção distratora D era muito popular entre os alunos. Essa opção distratora apresenta a condição de término do loop incorreta $j < x.length - 1$. Portanto, decidimos concentrar nossa análise nos alunos que primeiro eliminaram as opções A e B antes de decidir entre D e a opção correta, C.

5.5.1 Alunos do Quartil Superior

Havia quatro históricos escolares de alunos do quartil superior. Um desses alunos escolheu a opção B, e outro reduziu suas opções para B e D antes de escolher D. Essas duas transcrições serão ignoradas no restante desta análise. Dos dois alunos restantes, um escolheu corretamente a opção C e o outro escolheu a alternativa D. O aluno que respondeu corretamente utilizou uma explicação detalhada. Menos de 10% do texto da explicação, que tinha duas páginas, foi concluído quando esse aluno disse:

"... Então, vou seguir em frente e ir aprendendo com os erros." Vou igualar, vou colocar o A, a resposta A.

O aluno faz isso e prossegue de maneira semelhante à da transcrição na Figura 5.1. Aproximadamente 40% do caminho percorrido na transcrição de duas páginas, o aluno inicia a revisão novamente:

"Vou começar tudo de novo porque não anotei o suficiente."

O aluno prossegue como antes. Na transição da página 1 para a página 2 da transcrição, que ocorre no início desta segunda análise, o aluno faz uma inferência sobre o código da opção A: *"Na verdade, o problema de inserir esse código*

seria que ele sempre começa no início do array para o segundo valor, o valor de j. ... Então não pode ser A. Então agora estou tentando ... mas não vou me preocupar com B porque também começa no início do array para j. Então agora estou analisando C."

Logo no início da análise da opção C, o aluno ganha mais confiança:

"Não vejo nenhum motivo para que este não funcione, então vou tentar o último valor quando i é igual a 4. j é igual a 5, então 5 é menor que 6. Entramos no loop for. 1 é maior que 3? Não. Então j é igual a 6. 6 não é menor que 6 e i seria incrementado, e 5 não é menor que 5. Então parece que este funcionaria. Vou tentar D rapidinho. Vejo que a única diferença é que em j, no loop for com j, o comprimento é igual a, comparado a comprimento-1. Então vou tentar rapidinho, mas acho que vai acabar pulando uma posição. Então i é 0, 0 é menor que 5, verdadeiro. j é igual a 0 mais 1, então 1. 1 é menor que 5. Vou pular para onde j é 4. Então 4 é menor que 1? É verdade. E j seria O valor seria incrementado em 5, mas isso não passa no teste do loop for, então ele pularia o último valor da matriz, portanto D não funcionaria.

..."

Na transcrição acima, vemos sinais de uma compreensão emergente do conceito e da importância das condições de contorno.

O aluno restante, que restringiu a escolha à letra C e Antes de escolher incorretamente a opção D, D raciocinou muito mais rapidamente sobre sua escolha:

"Se você tentar ir até a posição 6, teremos um erro de índice fora dos limites da matriz, então queremos que vá até a posição 6-1, porque esse será o índice real do array."

É difícil entender como os alunos poderiam raciocinar incorretamente dessa forma na Questão 8, mas responder corretamente à questão seguinte, que também contém uma condição de término do laço. No entanto, como mostra a Tabela 3.3, apenas 51% dos alunos responderam corretamente à Questão 8, enquanto 73% responderam corretamente à Questão 9. Como especulamos anteriormente, talvez, ao responderem à Questão 8, os alunos se distraíam com a condição do laço na mesma página, no laço externo: $i < x.length - 1$.

5.5.2 Alunos do Segundo Quartil Na

análise anterior dos dados de desempenho, foi demonstrado que as opções C e D eram quase igualmente populares entre os alunos do segundo quartil.

alunos do quartil. Havia seis transcrições analisáveis desses alunos. Todos os seis alunos restringiram sua escolha à opção C ou D, e dois escolheram corretamente a opção C: 1) *“Tem que ser C ou D.*

Vai até o final?

Ou será que não? Mas na verdade deveria ir até o final, não é? Sim, deveria, então essa é a opção C.

2) *“...então o problema é, é $x.length$ ou $x.length-1$? Então, vai para $x.length-1$. Então você olha o número até o penúltimo número. O próximo deve olhar todos os números, deve olhar todos os números mais o último número que é... [indecifrável] ... Se você disser $x.length-1$, você não está passando por dois números, então esta é a resposta correta. C é a resposta correta”.*

Dos quatro alunos que escolheram a opção D, um admitiu ter chutado. As explicações oferecidas por dois dos três alunos restantes para a escolha da opção D parecem pouco diferentes das explicações dadas pelos alunos que escolheram a opção C: 1) *“E você precisa comparar até chegar ao final*

do array e o último elemento do array tiver comprimento -1...”.

2) *“... você quer ir até o final do array, então é $x.length-1$ ”.*

O aluno restante já havia sido citado anteriormente no artigo.

Este é o aluno que admitiu ter escolhido a opção D porque essa condição de loop era a mais semelhante à condição do loop externo:

“Sim, escolhi este porque achei que seria o mesmo, se você estiver olhando para o mesmo número de valores aqui.”

5.5.3 Alunos do Terceiro Quartil

Havia duas transcrições analisáveis desses alunos. Ambos restringiram sua escolha à opção C ou D, mas escolheram incorretamente D: 1) *“O último elemento é como*

comprimento-1 ... então é D.”

2) O aluno argumentou que, se a opção C fosse usada, *“... isso causaria um tipo de erro de segmentação”.*

5.5.4 Alunos do Quartil Inferior

Havia

um histórico escolar de um aluno do quartil inferior, e o aluno admitiu ter chutado a resposta.

5.5.5 Discussão dos dados de transcrição para a questão 8

Tal como na questão 2, poucas conclusões definitivas podem ser tiradas em relação à questão 8, devido ao pequeno número de transcrições que puderam ser analisadas.

Ao escolher entre as opções C e D, os alunos demonstram uma capacidade incipiente de raciocínio em um nível mais elevado. Nessa fase do desenvolvimento, muitos raciocinam incorretamente e escolhem a opção D. Dado que estavam raciocinando em um nível mais elevado, talvez não seja surpreendente que muitos alunos não tenham feito rabiscos ao tentar responder à questão 8.

Embora os alunos em todos os quartis tenham tido dificuldade em escolher entre as opções C ou D, os dados de desempenho mostram que os alunos nos três quartis superiores conseguiram eliminar consistentemente as opções A e B. Portanto, os alunos são capazes de algum grau de raciocínio analítico nessas questões de múltipla escolha com código resumido. No entanto, o fato de muitos alunos terem dificuldade em escolher corretamente a opção C em vez da D demonstra que essas habilidades analíticas são, neste estágio de desenvolvimento, frágeis.

6. DISCUSSÃO GERAL

Após analisarmos

todas as evidências disponíveis para este estudo, apresentamos agora algumas observações gerais.

6.1 Leitura versus Escrita

Surge a questão

de qual a relevância entre as tarefas de compreensão estudadas por este grupo de trabalho e a capacidade dos alunos de escrever código.

Mesmo quando nosso principal objetivo é ensinar os alunos a escrever código, exigimos que eles aprendam lendo código. Em nossas salas de aula, normalmente apresentamos exemplos de código aos alunos para ilustrar princípios gerais. Ao fazer isso, presumimos que nossos alunos consigam ler e entender esses exemplos. Quando incentivamos os alunos a ler o livro didático, presumimos que eles serão capazes de entender os exemplos contidos nele.

Perkins et al. (1989) afirmam que a capacidade de realizar uma inspeção visual é uma habilidade importante para diagnosticar erros e, portanto, a capacidade de revisar o código é uma habilidade importante na programação. Soloway (1986) afirma que, entre muitas outras habilidades, programadores experientes realizam frequentes “simulações mentais”, tanto de projetos abstratos em andamento quanto de código sendo aprimorado, como uma forma de evitar interações dinâmicas indesejadas entre os componentes do sistema. Ele argumenta que tais estratégias de simulação devem ser ensinadas explicitamente aos alunos. Muitas de nossas tradições de ensino remontam à era dos cartões perfurados. Nos tempos das execuções em lote noturnas, havia pouca necessidade de incentivar explicitamente os alunos a verificarem cuidadosamente seu código antes de submetê-lo para execução em lote, pois um erro por descuido poderia desperdiçar um dia inteiro. Em uma era em que a próxima execução de teste está a apenas um clique de distância, precisamos dar maior ênfase explícita à simulação mental como parte do processo de programação.

Wiedenbeck [1985] descobriu que programadores experientes executam atividades de programação de baixo nível mais rapidamente do que novatos. Tais atividades incluem identificar erros de sintaxe em uma única linha de código e avaliar a correção de laços com menos de 10 linhas de código. Wiedenbeck concluiu que os programadores experientes haviam “automatizado” esses processos, de modo que pouca atenção mental era necessária. Essa automação permitia que os especialistas se concentrassem em tarefas de resolução de problemas de nível superior. Como consequência dessas descobertas, Wiedenbeck sugeriu que o ensino de programadores novatos deveria “ênfatizar a prática contínua com materiais básicos” até que os novatos automatizem a prática [p. 389]. Portanto, talvez uma ênfase inicial na compreensão e no rastreamento de programas, com o objetivo de automatizar habilidades básicas, possa liberar a mente dos alunos para se concentrarem na resolução de problemas.

E, claro, em algum momento de suas carreiras, muitos programadores darão manutenção a programas escritos por outros, onde as habilidades de compreensão de programas são vitais [Deimel e Naveda, 1990].

6.2 Conceitos errôneos Spohrer

e Soloway [1986, 1989] coletaram dados sobre erros em programas escritos por novatos, onde os erros podiam ser atribuídos a conceitos errôneos sobre construções de programação. Eles concluíram que os erros devido a conceitos errôneos não eram tão disseminados quanto geralmente se acreditava.

Neste estudo, também observamos poucos erros de compreensão devido a concepções errôneas. Em apenas uma das doze questões de múltipla escolha houve uma concepção errônea frequente sobre um conceito de programação. Isso ocorreu na questão 6, em que os alunos não compreenderam completamente a semântica de “retornar”. Além disso, na transcrição de um aluno, que obteve 5 de 12 pontos, houve evidências de confusão entre a posição em um array e o conteúdo dessa posição. Portanto, é possível, e não seria surpreendente, que os alunos que obtiveram uma pontuação particularmente baixa nessas doze questões de múltipla escolha possam ter concepções errôneas.

Entre os 75% melhores alunos, em todas as questões, exceto uma, há pouca evidência de concepções errôneas sobre os conceitos. No entanto, as questões de múltipla escolha neste estudo utilizaram apenas fragmentos de código que realizavam processos iterativos em arrays. Os alunos podem ter concepções errôneas associadas a conceitos não abordados por essas questões de múltipla escolha.

6.3 Questões Não Idiomáticas Lembre-se de

que a Questão 2 é, na terminologia deste estudo, não idiomática. Ou seja, ela não processa toda a estrutura da frase.

O conteúdo dos dois arrays, como alguns programadores poderiam esperar inicialmente. Os dados de desempenho mostram que alunos com bom desempenho tendem a perceber esse detalhe não idiomático, enquanto alunos com mais dificuldades não. Até que ponto, então, questões de múltipla escolha como a Questão 2 são questões “capciosas”?

Como argumentaram Perkins et al. (1989), a capacidade de ler o que um trecho de código realmente faz, em vez do que poderíamos pensar em uma primeira inspeção rápida, é uma importante habilidade de depuração. Se os alunos forem avisados previamente de que os trechos de código nas questões de múltipla escolha podem ser não idiomáticos (ou conter erros, dependendo do ponto de vista), então consideramos essas questões legítimas. Uma limitação deste estudo é que apenas os alunos da instituição “base”, onde as questões foram elaboradas, tiveram a garantia de serem avisados previamente. No entanto, como os dados da instituição base são amplamente consistentes com os dados das instituições satélites, essa limitação não teve um impacto significativo no estudo.

Os autores deste grupo de trabalho reconhecem, no entanto, que uma fragilidade do código não idiomático é que ele incentiva os alunos a executar o código mecanicamente, quando também gostaríamos que eles lessem com o objetivo de abstrair para o esquema.

6.4 Significado e Contexto Ao ensinar

iniciantes a escrever programas, enfatizamos a importância de usar nomes de variáveis significativos. Eles são um tipo de farol [Brooks, 1983; Wiedenbeck, 1986] que ajuda o programador a entender um trecho de código. Existem também regras de discurso de programação [Soloway e Ehrlich, 1984] que estabelecem expectativas na mente dos programadores ao lerem o código. Além do próprio programa, observações de Pennington [1987] indicam que os programadores também usam uma estratégia de leitura de “referência cruzada”, na qual relacionam partes de um programa ao domínio do problema. Por exemplo, ao ler um programa que rastreia especificações de fiação de engenharia, um programador usa seu conhecimento prático de fiação.

As questões de múltipla escolha deste estudo tendem a não conter nomes de variáveis significativos, outros indicadores ou convenções de discurso. Além disso, essas questões não se relacionam a um domínio do mundo real. Portanto, em certa medida, essas questões são problemas artificiais, distantes, em certa medida, da tarefa de ler programas reais. No entanto, a maior parte do código nessas questões envolve algum tipo de operação plausível em arrays. Por exemplo, a questão 2 usa código para contar o número de elementos em comum em dois arrays (embora de forma não idiomática).

Embora a compreensão de programas reais possa envolver estratégias de nível superior usando indicadores, regras de discurso e referências cruzadas, essas habilidades não substituem a capacidade de rastrear sistematicamente o código. Detienne e

Soloway [1990] descobriu que, quando as habilidades de nível superior não conseguem elucidar o comportamento do programa, os programadores especialistas recorrem a outras habilidades, incluindo a simulação do programa. Além disso, os autores deste artigo acreditam que pode ser apropriado ensinar primeiro o traçado sistemático como uma habilidade básica e, em seguida, permitir que os alunos desenvolvam essas habilidades de compreensão de nível superior a partir dessa base.

6.5 Outras Linguagens Das doze

instituições participantes deste grupo de trabalho, onze ensinam Java como primeira linguagem e uma ensina C++. Essa restrição a apenas duas linguagens pode indicar sua popularidade como primeiras linguagens de programação. No entanto, também pode indicar que Java e C++ são particularmente difíceis de ensinar como primeiras linguagens. Seria interessante ver este estudo replicado usando outras linguagens de programação. Suspeitamos, porém, que o processo iterativo em arrays estudado neste artigo seja genérico a tal ponto que os alunos terão um desempenho semelhante, independentemente da linguagem que lhes for ensinada.

6.6 Dados Comparativos Grande

parte da literatura que estuda programadores iniciantes compara seu desempenho em uma determinada tarefa com o desempenho de programadores experientes. Como um estudo complementar, seria interessante coletar dados de programadores experientes para verificar como sua abordagem para responder a essas doze questões de múltipla escolha se compara à dos iniciantes estudados neste artigo.

6.7 Disponibilidade de Dados O grupo

de trabalho pretende, eventualmente, divulgar partes de seus dados para que outros possam realizar suas próprias análises. Informações sobre a disponibilidade de dados podem ser encontradas em um site [Lister, 2004].

7. CONCLUSÃO Este artigo é

um relatório de um grupo de trabalho do ITICSE 2004. Ele se baseia no trabalho anterior do grupo de trabalho McCracken. O grupo McCracken constatou que muitos alunos do primeiro ano de programação não conseguem programar ao final de seus cursos introdutórios. Embora uma explicação comum para essa incapacidade seja a de que os alunos não conseguem resolver problemas, no sentido estrito de cinco etapas definido pelo grupo McCracken, este grupo de trabalho constatou que muitos alunos carecem de conhecimentos e habilidades que são precursores da resolução de problemas. Esses elementos ausentes estão mais relacionados à capacidade dos alunos de ler código do que de escrevê-lo. Muitos dos alunos manifestaram uma

frágil capacidade de analisar sistematicamente um pequeno trecho de código.

Este grupo de trabalho não argumenta que todos os alunos que demonstram dificuldades na resolução de problemas o fazem devido a fatores relacionados à leitura. Reconhecemos que um aluno que obtém bom desempenho nos tipos de testes utilizados neste estudo, mas que não consegue escrever código inédito de complexidade semelhante, provavelmente apresenta alguma dificuldade na resolução de problemas.

Este grupo de trabalho apenas observa que qualquer projeto de pesquisa que vise estudar as habilidades de resolução de problemas em programadores iniciantes deve incluir um mecanismo para identificar indivíduos com deficiências em habilidades precursoras relacionadas à leitura.

Este estudo partiu do pressuposto de que o conhecimento e as habilidades que são o foco desta pesquisa são precursores da resolução de problemas. O próximo passo lógico da pesquisa é examinar essa suposição, combinando os delineamentos do estudo de McCracken com os deste estudo, pedindo aos alunos que resolvam tarefas semelhantes às deste estudo e também que escrevam código de complexidade similar.

Agradecimentos Oito dos doze

autores começaram a discutir este estudo durante os projetos "Bootstrapping" e "Scaffolding", conduzidos por Sally Fincher, Marian Petre e Josh Tenenber.

Esses projetos foram financiados pela Fundação Nacional de Ciência (nº DUE-0122560). Quaisquer opiniões, conclusões ou recomendações expressas neste material são de responsabilidade dos autores e não refletem necessariamente a posição da Fundação Nacional de Ciência. Em algumas instituições, os autores deste artigo não ministraram as aulas nas quais os alunos fizeram os testes de múltipla escolha.

Agradecemos, portanto, a colaboração dos professores dessas turmas: David Bernstein (James Madison), Kerttu Pollari-Malmi (Helsinki), Mohamed Kerasha (Connecticut), Roger Simons e Ying Zhou (Rhode Island). Os autores agradecem a Mike McCracken por seus conselhos e apoio, e também aos organizadores da conferência de Leeds por disponibilizarem excelentes instalações ao grupo de trabalho. Por fim, os autores agradecem a George Burdell por seus comentários pertinentes sobre uma versão preliminar deste artigo.

REFERÊNCIAS

- Brooks, R. (1983) Rumo a uma teoria da compreensão de programas de computador. *International Journal of man-Machine Studies*, **18**, pp. 543-554.
- Clancy, M. e Linn, M. (1999), Padrões e Pedagogia. 30º Simpósio Técnico sobre Educação em Ciência da Computação (SIGCSE 1999), Nova Orleans, LA EUA. pp. 37-42.

- Davies, S. (1996) Estratégias de resolução de problemas baseadas em exibição na programação de computadores. Em Gray, W, e Boehm-Davis, D. (Eds) Estudos empíricos de programadores: 6º workshop. Ablex Publishing Corporation, NJ. pp. 59-76.
- Detienne, F. (1990) Conhecimento especializado em programação: uma abordagem baseada em esquemas. Em Hoc, J, Green, T, Samurcay e Gilmore, D. (Eds) Psicologia da programação. Academic Press, Londres. pp 206-222.
- Detienne, F, e Soloway, E. (1990) Uma estrutura de controle derivada empiricamente para o processo de compreensão de programas. *Int. J. of Man-Machine Studies*, **33**, pp. 323-342.
- Deimel, LE & Naveda, JF (1990) Leitura de Programas de Computador: Guia do Instrutor e Exercícios. Instituto de Engenharia de Software, Universidade Carnegie Mellon. http://www.deimel.org/comp_sci/reading_computer_p_programas.htm (agosto de 2004) Bibliografia atualizada em <http://www2.umassd.edu/SWPI/ProcessBibliography/bib-codereading2.html> (agosto de 2004).
- du Boulay, B. (1989) Algumas dificuldades de aprender a programar. Em Soloway, E. e Spohrer, J., Eds. pp 283-299.
- Ebel, R. e Frisbie, D. (1986) Fundamentos da Medição Educacional. Prentice Hall, Englewood Cliffs, NJ.
- Haladyna, T. (1999) Desenvolvimento e Validação de Questões de Múltipla Escolha (2ª Edição), Lawrence Erlbaum Associates, Mahwah, NJ.
- Kuittinen, M, e Sajaniemi, J. (2004) Ensinando os papéis das variáveis em cursos de programação elementar. 9ª Conferência Anual SIGCSE sobre Inovação e Tecnologia no Ensino de Ciência da Computação (ITiCSE'04), Leeds, Reino Unido. pp 57-61.
- Linn, R. e Gronlund, N. (1995) Medição e avaliação no ensino, . Prentice Hall, Upper Saddle River, NJ.
- Lister, R. (2004) Disponibilidade de dados do grupo de trabalho. <http://www-staff.it.uts.edu.au/~raymond/leeds2004>.
- McCracken, M., V. Almstrum, D. Diaz, M. Guzdial, D. Hagen, Y. Kolikant, C. Laxer, L. Thomas, I. Utting, T. Wilusz, (2001) Um estudo multinacional e multi-institucional da avaliação das habilidades de programação de alunos do primeiro ano de Ciência da Computação, SIGCSE Bulletin, 33(4). pp 125-140.
- Pennington, N. (1987) Estratégias de compreensão em programação. Em Olson, G., Sheppard, S., e Soloway, E. (Eds) Estudos empíricos de programadores: Segundo workshop. Ablex, NJ, EUA. pp 100-113.
- Perkins, D. e Martin, F. (1986) Conhecimento frágil e estratégias negligenciadas em programadores novatos. Em Soloway, E. e Iyengar, S. (Eds) pp. 213-229.
- Perkins, D, Hancock, C, Hobbs, R, Martin, F, e Simmons, R. (1989). Condições de aprendizagem em programadores novatos. Em Soloway, E. e Spohrer, J., Eds. pp 261-279.
- Rist, RS (1986). Planos em Programação: Definição, Demonstração e Desenvolvimento. Em Soloway, E. e Iyengar, S., Eds. pp 28-47.
- Rist, R. (2004) Aprendendo a programar: Criação, aplicação e avaliação de esquemas. Em Fincher, S e Petre, M., Eds (2004) Pesquisa em educação em ciência da computação. Swets & Zeitlinger.
- Soloway, E. e Ehrlich, K (1984) Estudos empíricos do conhecimento de programação. *IEEE Transactions on Software Engineering*, SE-10(5):595-609.
- Soloway, E. (1986). Aprender a programar = Aprender a dar construir mecanismos e explicações. *Comunicações da ACM*, 29(9). pp. 850-858.
- Soloway, E. e Iyengar, S., Eds (1986) Estudos empíricos de programadores. Ablex, NJ, EUA.
- Soloway, E., Adelson, B. e Ehrlich, K. (1988) Conhecimento e processos na compreensão de programas de computador. Em Glaser, M, Chi, R, Farr, M, Glaser, R (Eds) A natureza da especialização. Lawrence Erlbaum Associates, Hillsdale, NJ, EUA. pp 129-152.
- Soloway, E. e Spohrer, J, Eds (1989), Estudando o Programador Novato. Lawrence Erlbaum Associates, Hillsdale, NJ, 1989.
- Spohrer, J. e Soloway, E. (1986) Analisando os erros de alta frequência em programas iniciantes. Em Soloway, E. e Iyengar, S. (Eds) pp. 230-251.
- Spohrer, J. e Soloway, E. (1989) Erros de novatos: as sabedorias populares estão corretas? Em Soloway, E. e Spohrer, J., Eds. pp 401-416.
- Thomas, L. Ratcliffe, M., e Thomasson, B. (2004) Estruturação com diagramas de objetos em aulas de programação do primeiro ano: alguns resultados inesperados. 35º Simpósio Técnico sobre Educação em Ciência da Computação (SIGCSE 2004), Norfolk, VA, EUA. pp. 250-254.
- Wiedenbeck, S. (1985) Diferenças entre novatos e especialistas em habilidades de programação. *Int. J. of Man-Machine Studies*, **23**, pp. 383-390.
- Wiedenbeck, S. (1986) Processos na compreensão de programas de computador. Em Soloway, E. e Iyengar, S., Eds. pp 48-57.

Apêndice A: As 12 questões de múltipla escolha.

As questões foram apresentadas aos alunos em formato de coluna única, geralmente com uma questão completa por página. Aqui, o recuo de algumas perguntas foi alterado para se adequar ao formato da revista. As respostas para as perguntas são apresentadas após a última pergunta.

Questão 1

Considere o seguinte fragmento de código:

```
int[] x = {2, 1, 4, 5, 7}; int limit = 3; int i = 0; int sum
= 0; while ( (sum<limit) &&
(i<x.length)){ ++i;
sum += x[i];

}
```

Qual o valor da variável "i" após a execução deste código? a) 0 b) 1 c) 2 d) 3

Pergunta 2.

Considere o seguinte fragmento de código:

```
int[] x1 = {1, 2, 4, 7}; int[] x2 = {1, 2, 5, 7}; int
i1 = x1.length-1; int i2 = x2.length-1; int count
= 0; while ((i1 > 0) && (i2 > 0)) {
```

```
    se ( x1[i1] == x2[i2] ) {

        ++contagem; --
        i1; --i2;

    } senão se (x1[i1] < x2[i2]) {

        --i2;

    } outro
    {        // x1[i1] > x2[i2] --i1;

    }

}
```

Após o término do loop while acima, qual valor será o valor de "count"? a) 3 b) 2 c) 1 d) 0

Pergunta 3.

Considere o seguinte fragmento de código:

```
int [] x = {1, 2, 3, 3, 3}; boolean b[] = new
boolean[x.length];

para (int i = 0; i < b.length; ++i) b[i] = falso;

para (int i = 0; i < x.length; ++i) b[x[i]] = verdadeiro;

int count = 0;

para (int i = 0; i < b.length; ++i) {

    se ( b[i] == verdadeiro ) ++contagem;

}
```

Após a execução deste código: , "contagem" contém:

- a) 1
- b) 2
- c) 3
- d) 4
- e) 5

Questão 4.

Considere o seguinte fragmento de código:

```
int[] x1 = {0, 1, 2, 3}; int[] x2 = {1, 2, 2, 3}; int i1
= 0; int i2 = 0; int count = 0; while ( (i1 <
x1.length) && (i2 <
x2.length))
```

```
{

    se ( x1[i1] == x2[i2] ) {

        ++contagem; +
        +i2;

    } senão se (x1[i1] < x2[i2]) {

        ++i1;

    } outro
    {        // x1[i1] > x2[i2] ++i2;

    }

}
```

Após a execução deste código, "count" contém: a) 0 b) 1 c) 2 d) 3 e) 4

Pergunta 5.

Considere o seguinte fragmento de código:

```
int[] x = {0, 1, 2, 3}; int temp; int i = 0; int j =
x.length-1;
```

```
enquanto (i < j) {
```

```
    temp = x[i]; x[i] = x[j];
    x[j] = 2*temp; i++; j--;
```

```
}
```

Após a execução deste código, o array “x” conterá os seguintes valores:

a) {3, 2, 2, 0} b)
{0, 1, 2, 3} c) {3,
2, 1, 0} d) {0, 2,
4, 6} e) {6, 4, 2, 0}

Questão 6.

O método “isSorted” deve retornar verdadeiro se o array estiver ordenado em ordem crescente. Caso contrário, o método deve retornar falso:

```
public static boolean isSorted(int []x) {

    //O código que falta vai aqui

}
```

Qual das seguintes opções representa o código que falta no método “isSorted”?

(a) booleano b = verdadeiro;

```
para (int i=0 ; i<x.length-1; i++) {

    se ( x[i] > x[i+1] ) b = falso;

    outro

        b = verdadeiro;

} retornar b;
```

(b) para (int i=0; i<x.length-1; i++) {

```
    se (x[i] > x[i+1]) retorne falso;
```

```
} retornar verdadeiro;
```

(c) booleano b = falso;

```
para (int i=0; i<x.length-1; i++) {

    se (x[i] > x[i+1] )

        b = falso;

} retornar b;
```

(d) booleano b = falso;

```
para (int i=0;i<x.length-1;i++) {

    se (x[i] > x[i+1] ) b = verdadeiro;

} retornar b;
```

(e) para (int i=0;i<x.length-1;i++) {

```
    se (x[i] > x[i+1] ) retorne verdadeiro;
```

```
} retornar falso;
```

Questão 7.

Considere o seguinte fragmento de código:

```
int[] x = {2, 1, 4, 5, 7}; int limite = 7; int i = 0; int
soma = 0;
```

```
enquanto ( (soma<limite) && (i<x.comprimento) ) {

    soma += x[i]; ++i;

}
```

Qual o valor da variável “i” após a execução deste código?

a) 0
b) 1
c) 2
d) 3
e) 4

Pergunta 8.

Se dois números quaisquer em uma matriz de inteiros, não necessariamente consecutivos, estiverem fora de ordem (ou seja, o primeiro número na matriz é maior que o segundo), isso é chamado de inversão. Por exemplo, considere uma matriz "x" que contém os seguintes seis números:

4 5 6 2 1 3

Existem 10 inversões nesse arranjo, como:

```
x[0]=4 > x[3]=2 x[0]=4 > x[4]=1
x[0]=4 > x[5]=3 x[1]=5 > x[3]=2
x[1]=5 > x[4]=1 x[1]=5 > x[5]=3
x[2]=6 > x[3]=2 x[2]=6 > x[4]=1
x[2]=6 > x[5]=3 x[3]=2 > x[4]=1
```

O código-fonte abaixo tem como objetivo contar o número de inversões em um array "x":

```
int inversionCount = 0;
```

```
para (int i=0; i<x.length-1; i++) {
```

```
    para xxxxxx
    {
        se ( x[i] > x[j] ) ++contagem de
                               inversões;
    }
}
```

Ao término da execução do código acima, a variável "inversionCount" deverá conter o número de inversões no array "x". Portanto, o "xxxxxx" no código acima deve ser substituído por:

a) (int j=0; j<x.length; j++)

b) (int j = 0; j < x.length - 1; j++) c) (int j = i + 1; j < x.length; j++) d)

(int j = i + 1; j < x.length - 1; j++)

Questão 9.

O código abaixo tem como objetivo copiar para um array de inteiros chamado "array2" quaisquer números pares presentes em outro array de inteiros chamado "array1". Por exemplo, se "array1" contivesse os números:

array1: 4 5 6 2 1 3

Após o processo de cópia, "array2" deverá conter, em suas três primeiras posições:

array2: 4 6 2

O código a seguir pressupõe que "array2" seja grande o suficiente para armazenar todos os números pares de "array1":

```
int a2 = 0;
```

```
para (int a1 = 0; xxx1xxx; ++a1) {
```

```
    // se array1[a1] for par if ( array1[a1] % 2 ==
    0 ) {
```

```
        // array1[a1] é par, // então copie-o
        xxx2xxx; xxx3xxx;
```

```
    }
```

```
}
```

Os trechos de código ausentes "xxx1xxx", "xxx2xxx" e "xxx3xxx" no código acima devem ser substituídos respectivamente por:

um) a1<array1.length ++a2

```
array2[a2] = array1[a1]
```

b) a1<matriz1.comprimento
matriz2[a2] = matriz1[a1] ++a2

c) a1<=array1.comprimento
array2[a2] = array1[a1] ++a2

d) a1<=array1.length ++a2

```
array2[a2] = array1[a1]
```

Dica: em todas as quatro opções acima, a segunda e a terceira partes são iguais, apenas invertidas.

Questão 10.

Considere o seguinte fragmento de código:

```
int[] matriz1 = {2, 4, 1, 3}; int[] matriz2 = {0, 0, 0, 0}; int a2 = 0;

para (int a1=1; a1<array1.length; ++a1) {

    se ( array1[a1] >= 2 ) {

        matriz2[a2] = matriz1[a1]; ++a2;

    }

}
```

Após a execução deste código, quais valores conterá o array "array2"?

- a) {4, 3, 0, 0} b) {4, 1, 3, 0} c) {2, 4, 3, 0} d) {2, 4, 1, 3}

Questão 11.

Suponha que um vetor de inteiros "s" contenha zero ou mais inteiros positivos diferentes, em ordem crescente, seguidos por um zero. Por exemplo:

```
int[] s = {2, 4, 6, 8, 0};
ou int[] s = {0};
```

Considere o seguinte código "esqueleto", onde as sequências de "xxxxxx" são substituídas para os valores corretos. Código Java:

```
int pos = 0; while
( (xxxxxx) && (xxxxxx) ) ++pos;
```

Suponha que uma variável inteira "e" contenha um número inteiro positivo. O objetivo do código acima é encontrar a posição em "s" ocupada pelo valor armazenado em "e". Formalmente, quando o laço "while" acima termina, a variável "pos" é determinada da seguinte forma:

1. Se o valor armazenado em "e" também estiver armazenado na matriz, então "pos" contém o índice dessa posição. Por exemplo, se e = 6 e s = {2, 4, 6, 8, 0}, então pos deve ser igual a 2.
2. Se o valor armazenado em "e" NÃO estiver armazenado na matriz, mas o valor em "e" for menor que alguns dos valores na matriz, então "pos" contém o índice da posição mais baixa na matriz onde o valor é maior que em

"e". Por exemplo, se e=7 e s = {2, 4, 6, 8, 0}, então pos deve ser igual a 3.

3. Se o valor armazenado em "e" for maior que qualquer valor em "s", então "pos" contém o índice da posição que contém o zero. Por exemplo, se e = 9 e s = {2, 4, 6, 8, 0}, então pos deve ser igual a 4.

A condição booleana correta para o loop "while" acima é: (a) (pos < e)

&& (s[pos] != 0) (b) (pos != e) && (s[pos] != 0) (c) (s[pos] < e) && (pos != 0) (d) (s[pos] < e) && (s[pos] != 0) (e) (s[pos] != e) && (s[pos] != 0)

Questão 12.

Esta questão dá continuidade à questão anterior.

Supondo que tenhamos encontrado a posição no array "s" que contém o mesmo valor armazenado na variável "e", agora desejamos escrever um código que remova esse número do array, mas mantenha a ordem crescente de todos os demais inteiros no array. Por exemplo, dado:

s = {2, 4, 6, 8, 0}; e = 6; pos = 2;

O objetivo é remover o 6 de "s" para obter:

s = {2, 4, 8, 0, 0};

Considere o seguinte código "esqueleto", onde "xxxxxx" é um substituto para o código Java correto:

```
fazer {
    ++pos;
    xxxxxx;
} enquanto (s[pos] != 0);
```

A substituição correta para "xxxxxx" é:

- (a) s[pos+1] = s[pos]; (b) s[pos] = s[pos+1]; (c) s[pos] = s[pos-1]; (d) s[pos-1] = s[pos]; (e) Nenhuma das alternativas acima.

Opções corretas para as 12 questões: 1 2 3

4 5 6 7 8 9 10 11 12
cbceabdcbbdd

Apêndice B. Análise das questões individuais

Este apêndice contém os resultados estatísticos e a análise de cada uma das questões de múltipla escolha. As análises das questões 2 e 8, encontradas no corpo do texto, não são repetidas. Questões com estrutura semelhante estão agrupadas nas seções B.1 a B.3.

B.1 Questões de código fixo com respostas do tipo

int As questões 1 a 4 e 7 fornecem um array e pedem o valor de uma variável do tipo `int`, que representa o número de vezes que um elemento ocorre no array ou o índice de uma posição no array. Teoricamente, existe um conjunto finito de respostas possíveis – o conjunto de índices do array fornecido ou o intervalo de 0 até o número total de elementos no array.

Considere a Questão 1, por exemplo. Esta questão envolve percorrer um array, acumular a soma dos elementos do array em uma variável e registrar o índice no qual a soma atinge um determinado limite. Há cinco elementos no array, portanto, os valores de índice possíveis incluem 0, 1, 2, 3 e 4. Se o valor do índice for incrementado ao final do loop, ele também poderá ser 5. Nem todos esses valores são fornecidos como opções: as opções incluem apenas 0, 1, 2 e 3.

As respostas dos alunos a esta questão estão resumidas no gráfico da Figura B1, abaixo. Este gráfico mostra que os alunos com melhor desempenho (aqueles no primeiro quartil) tiveram um bom resultado; mais de 90% deles escolheram a alternativa C (a resposta correta). Embora o segundo e o terceiro quartis não tenham um desempenho tão bom, eles ainda demonstram uma forte preferência pela resposta correta. A questão distingue entre os quartis 1 a 3 e os alunos com pior desempenho, que escolhem a resposta correta e a alternativa D com frequência aproximadamente igual.

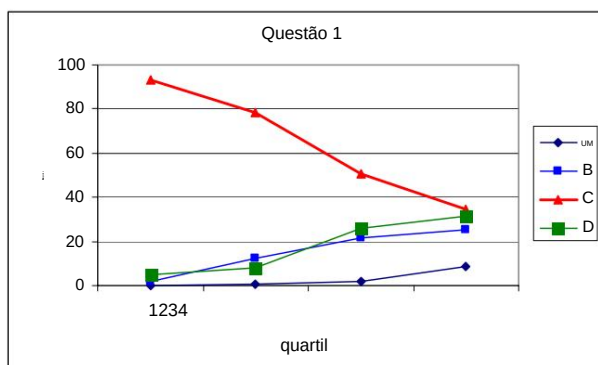


Figura B1: Respostas dos alunos à Pergunta 1 por quartis.

Mesmo os alunos do quarto quartil sabem de algo, no entanto: nem todas as opções são igualmente populares. A opção A é muito menos popular que as demais, mesmo entre os alunos do quarto quartil. O valor da opção A é 0, e nossa hipótese é que mesmo os alunos com mais dificuldades geralmente conseguem perceber que `i++` altera o valor de `i` pelo menos uma vez.

A discussão da Questão 2 encontra-se na Seção 4.1.

A questão 3, outra questão de código fixo, envolve a manipulação de dois arrays. Nesse caso, o segundo array, um array de booleanos, é usado para controlar os valores distintos contidos no primeiro array (de inteiros). Finalmente, o número de valores distintos no array de inteiros (cada um correspondendo a verdadeiro no array de booleanos) é contado.

O número de elementos na matriz é 5, portanto, os valores possíveis para o número de elementos distintos são 1, 2, 3, 4 e 5. Todos esses valores são fornecidos como opções.

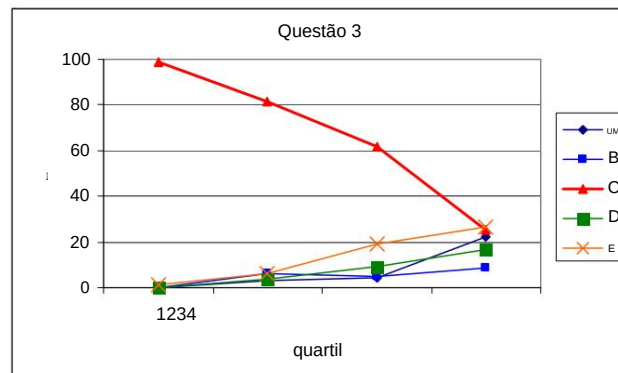


Figura B2: Respostas dos alunos à Pergunta 3 por quartis.

A julgar pelos resultados, os alunos tiveram pouca dificuldade com esta questão. Mais uma vez, os melhores alunos se saíram muito bem, e os quartis 2 e 3 demonstraram uma forte preferência pela resposta correta. A alternativa E torna-se cada vez mais popular à medida que descemos dos melhores alunos para os alunos com mais dificuldades. A alternativa E, "5", é a que um aluno escolheria caso não compreendesse o enunciado.

`b[x[i]] = verdadeiro;`

e simplesmente assumiu que era o mesmo que

`b[i] = verdadeiro;`

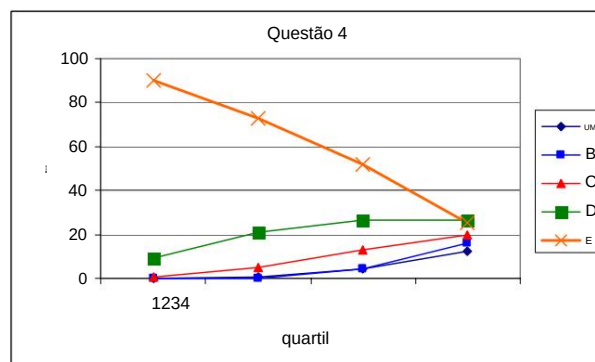


Figura B3: Respostas dos alunos à questão 4 por quartis.

A questão 4, também de código fixo, é muito semelhante à questão 2. Ela também envolve a comparação de dois arrays, procurando elementos correspondentes. Novamente, cada array contém quatro elementos.

Existem três diferenças entre as questões 2 e 4: 1. A questão 4 começa na posição 0 e incrementa o índice do array; a questão 2 começa na posição final do array e decrementa o índice.

2. A questão 4 examina todas as posições na matriz; a questão 2 examina todas, exceto uma.
3. Quando a Questão 4 encontra um par correspondente, ela altera apenas um dos índices; a Questão 2 altera ambos. Assim, por exemplo, suponha que temos dois arrays: {6, 0} e {6, 6}. O código da Questão 4 encontra duas correspondências; o código da Questão 2 encontra apenas uma.

Devido a essas diferenças, as respostas possíveis aqui variam de 0 (por exemplo, {1, 2, 3, 4} e {5, 6, 7, 8}) a 4 (por exemplo, {2, 3, 4, 5} e {2, 2, 2, 2}).

As opções fornecidas para esta questão estão relacionadas às concepções errôneas mais prováveis dos alunos. A resposta correta é 4 (opção E). Se você acha que este código está contando duplicatas da mesma forma que a Questão 2, você chegaria à resposta 3 (que é a opção D). Se você perceber que o código está contando de forma ligeiramente diferente, mas achar que ele ainda está ignorando o primeiro elemento de cada array, você também chegaria à resposta 3. Se você cometer ambos os erros, você chegaria à resposta 2 (opção C).

No gráfico da Questão 4, vemos o padrão familiar: os melhores alunos têm um desempenho muito bom, os alunos dos quartis 2 e 3 têm um desempenho inferior, mas ainda demonstram uma forte preferência pela resposta correta, e o quartil 4 tem um desempenho ainda pior e não demonstra preferência pela resposta correta.

Observamos também, no entanto, que todos os quatro quartis distinguem entre as alternativas incorretas, sendo a alternativa D a mais popular, seguida pela alternativa C, o que está de acordo com o fato de que as prováveis concepções errôneas levam a essas respostas. As alternativas A e B, que não correspondem a nenhuma das prováveis concepções errôneas, são as menos populares. E, mais uma vez, 0 (neste caso, a alternativa A) é uma opção muito impopular.

Segue o gráfico da questão 7:

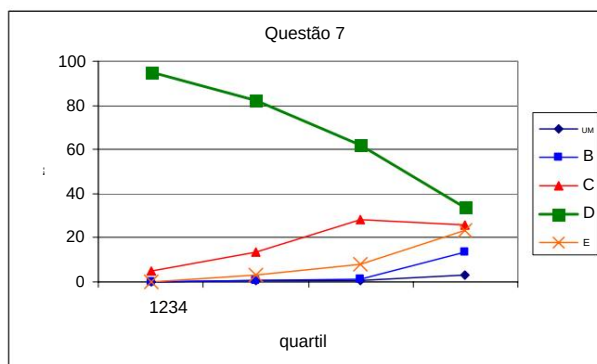


Figura B4: Respostas dos alunos à questão 7 por quartil.

A questão 7 é muito semelhante à questão 1. Ambas as questões envolvem um único array. Em ambos os casos, o código percorre o array, acumulando a soma dos elementos e registrando o índice em que a soma atinge um determinado limite. O array fornecido na questão 1 é idêntico ao array fornecido na questão 7 e, em ambos os casos, o código começa na posição 0 do array e percorre os elementos em ordem crescente ou decrescente. Em ambos os casos, como o array possui cinco elementos, os valores possíveis para o índice são 0, 1, 2, 3 e 4 (e 5, se o índice for incrementado após ser utilizado).

Existem três diferenças entre as perguntas:

1. O valor do limite (3 para a questão 1 e 7 para a questão 7).
2. A ordem dos passos dentro do laço while. Na Questão 1, o índice é incrementado primeiro e, em seguida, o elemento é adicionado à soma; na Questão 7, o elemento é adicionado primeiro e, em seguida, o índice é incrementado.
3. As opções para a questão 7 incluem 4 (opção E), bem como 0, 1, 2 e 3.

Mais uma vez, os alunos com melhor desempenho se saíram muito bem, enquanto os alunos nos quartis 2 e 3 tiveram um desempenho inferior, mas demonstraram uma forte preferência pela resposta correta. Neste caso, os alunos do quartil 4 também apresentaram uma leve preferência pela opção correta. Houve relativamente poucos erros, e as transcrições indicam que alguns deles ocorreram devido a erros de transcrição, e não por falta de compreensão.

Em resumo, todas essas questões são semelhantes: todas nos ajudam a distinguir entre os três quartis superiores, por um lado, e os alunos com pior desempenho, por outro. Seus gráficos de linhas de resposta são parecidos, todas são questões fáceis para os melhores alunos e, em cada caso, os alunos do quarto quartil são os únicos que não demonstram uma forte preferência pela resposta correta. Às vezes, como na Questão 2, podemos observar evidências de que os alunos presumem que o código se encaixa em expressões idiomáticas que conhecem (como processar arrays a partir de 0). E o 0 sempre parece ser um distrator inadequado.

B.2 Questões de código fixo com respostas em formato de matriz

As questões 5 e 10 também são questões de código fixo e fornecem um array e um código que processa esse array. No entanto, em vez de um valor inteiro, elas pedem o array resultante.

Segue o gráfico da Questão 5:

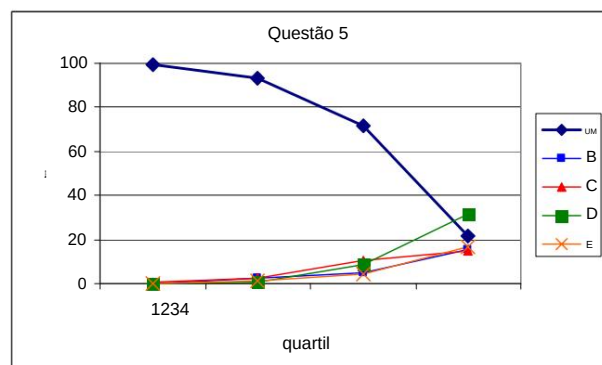


Figura B5: Respostas dos alunos à questão 5 por quartis.

O código para a Questão 5 pega um array e inverte a ordem dos elementos no array, exceto por um detalhe: os elementos que estavam na primeira metade do array são duplicados antes de serem movidos para suas novas posições na segunda metade do array. Assim, o array {1, 3, 5, 7} se tornaria {7, 5, 6, 2}.

As opções estão relacionadas a possíveis equívocos dos alunos. Elas incluem:

o O array original o O array

que você obterá se invertesse o array original, mas não duplicasse nenhum de seus elementos

O array que você obterá se invertesse o array original e dobrasse todos os seus elementos. O array que você obterá se dobrasse todos os elementos do array, mas não o invertesse. O correto.

responder.

Esta questão distingue ainda mais claramente do que as questões anteriores os três quartis superiores dos alunos com pior desempenho. Foi uma das questões mais fáceis para os três quartis superiores, como mostra o gráfico: a linha de resposta correta tem um formato semelhante, mas é muito alta, principalmente para o segundo e o terceiro quartis.

Mais de 95% dos alunos do segundo quartil escolheram a resposta correta.

Mesmo no terceiro quartil, mais de 70% dos alunos escolheram a resposta correta.

O primeiro e o segundo quartis não mostram uma preferência clara por nenhuma das alternativas incorretas; o terceiro quartil prefere ligeiramente a alternativa D (dobrar toda a matriz sem invertê-la), e o quarto quartil prefere a alternativa D até mesmo à resposta correta. As anotações dos alunos nas provas indicam que mesmo os alunos com mais dificuldades conseguem perceber que algo está sendo multiplicado por dois, mas têm mais dificuldade em acompanhar a lógica dos elementos trocados.

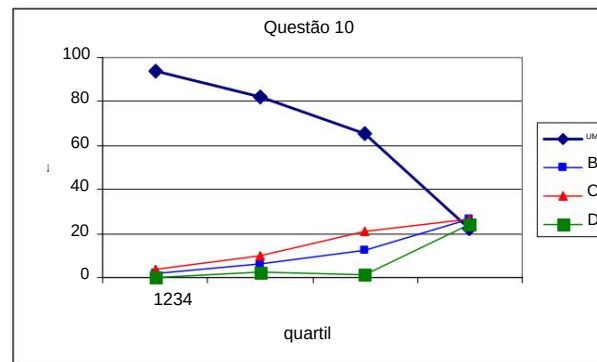


Figura B6: Respostas dos alunos à questão 10, por quartis.

A questão 10 é a última das questões de código fixo. Ela envolve filtrar um array: pegar todos os números maiores ou iguais a dois e copiá-los para um segundo array.

O primeiro elemento da matriz é ignorado.

As opções, mais uma vez, estão relacionadas a possíveis equívocos dos alunos. Elas incluem:

o array com todos os elementos do array original que são maiores ou iguais a dois (o que você obterá se filtrasse, mas não percebesse que a cópia começou na posição 1)

o array que você obterá se começasse com posição 1, mas não filtrou

o array que você obterá se não filtrasse e Não percebi que a cópia começou na posição 1 (ou seja, no array original).

ou a resposta correta.

No geral, 70% dos alunos resolveram esta questão corretamente, e os alunos do primeiro quartil preferiram esmagadoramente a resposta correta (A). Os alunos do segundo e terceiro quartis também preferiram fortemente a resposta correta. Os alunos com mais dificuldades, por outro lado, evidentemente chutaram: escolheram todas as respostas com igual frequência.

Todos, exceto o quartil inferior, perceberam algo errado: a opção D é a que contém os dois erros possíveis e, como mostra a Figura 10, é muito impopular em todos os quartis 1 a 3. Nos quartis 2 e 3, a opção C (a resposta obtida se você assumir que o processamento da matriz inclui a matriz inteira) é um pouco mais popular do que a opção B (a resposta obtida se você perceber que o primeiro elemento da matriz está sendo ignorado, mas você não filtra a matriz).

Os alunos do quarto quartil parecem estar escolhendo quase aleatoriamente, talvez porque não tenham expressões idiomáticas em que se basear.

Em resumo, as questões de código fixo têm muito em comum. Seus gráficos de trajetória são muito semelhantes: os melhores alunos têm um desempenho excelente, os alunos do segundo e terceiro quartis têm um desempenho inferior, mas ainda assim demonstram uma forte preferência pela resposta correta.

resposta, e os alunos no quarto quartil não têm preferência pela resposta correta. Às vezes, podemos tirar conclusões com base nas alternativas incorretas escolhidas pelos alunos. Essa evidência tende a corroborar a ideia de que muitos alunos são influenciados por expressões idiomáticas padrão, como percorrer um array da posição 0 até a posição de comprimento -1, mas ainda não se sentem suficientemente à vontade com essas expressões para saber quando não usá-las.

B.3 Questões de código esqueleto

Neste ponto, o leitor pode estar se perguntando se os gráficos de linhas de rastreamento sempre têm a mesma aparência. Duas das questões de código esqueleto foram relativamente fáceis para os alunos, e seus gráficos são semelhantes aos gráficos das questões de código fixo. As outras três questões foram consistentemente as mais difíceis, em todos os quartis e em todas as instituições, e seus gráficos são bastante diferentes. A questão 8, uma questão de código esqueleto difícil, foi apresentada na seção 4, portanto não será apresentada aqui.

Vamos começar com uma pergunta mais fácil. Aqui está o gráfico da Pergunta 9:

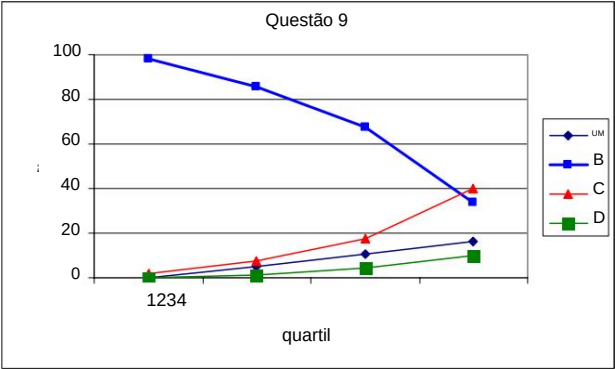


Figura B7: Respostas dos alunos à questão 9 por quartis.

Nesta questão, os alunos são informados de que um determinado fragmento de código tem como objetivo copiar os números pares contidos em uma matriz para uma segunda matriz; eles devem preencher três lacunas no código.

As opções são claramente paralelas e, caso o layout não seja suficiente para deixar isso claro para os alunos, uma dica é fornecida.

A atenção dos alunos está focada em dois pontos:

- 1. O loop deve parar de processar o array quando o índice for igual ao comprimento do array, ou quando for igual ao comprimento menos 1?
- 2. O índice do segundo array deve ser incrementado no início ou no final do corpo do loop?

A resposta correta para ambas as perguntas era a idiomática: incrementar no final do corpo do loop e parar o processamento na posição de comprimento -1.

As opções incluíam:

- o Interrompa o processamento no local correto, mas incremente no início do corpo do loop. o Interrompa
- o processamento no local correto e incremente no final do corpo do loop (a resposta correta).
- o Interromper o processamento no local errado e incrementar no início do corpo do loop (dois erros combinados)
- o Interrompa o processamento no lugar errado, mas incremente no final do corpo do loop

Esta foi a questão mais fácil entre as questões de código esqueleto e uma das mais fáceis no geral. Do quartil superior, 98% acertaram a questão. O que é ainda mais impressionante é que mesmo os alunos com mais dificuldades conseguiram diferenciar entre as respostas em que o contador do loop era incrementado no início (B e C) e aquelas em que era incrementado no final (A e D), favorecendo a ordem correta das instruções. Considerando o número de alunos que chutaram em algumas das questões, isso se torna ainda mais notável.

O bom desempenho dos alunos nesta questão pode ser atribuído ao fato de a resposta idiomática e previsível ser, de fato, a correta. Pode também ser porque as respostas são claramente distintas entre si, direcionando a atenção dos alunos para os pontos-chave. É possível ainda que a Questão 10 tenha influenciado o resultado. Um aluno atento poderia encontrar a resposta da Questão 9 na Questão 10. A Questão 10 também envolve filtrar um array para obter outro, mas trata-se de uma questão de código fixo, portanto o código é fornecido. Por outro lado, é discutível se um aluno suficientemente atento para perceber a semelhança entre as duas questões teria acertado ambas.

Para uma questão bem diferente, aqui está o gráfico da Questão 6:

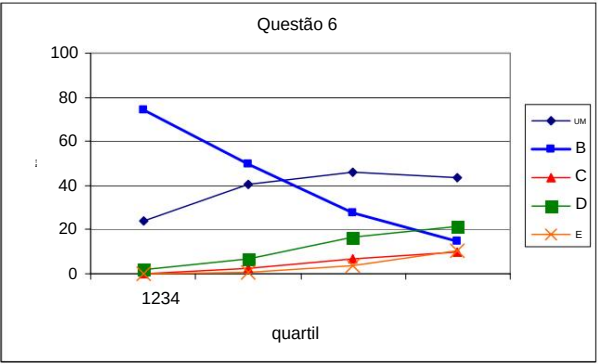


Figura B8: Respostas dos alunos à questão 6, por quartis.

Esta questão pede ao aluno que selecione o corpo do método para uma função booleana que recebe um array de inteiros e retorna verdadeiro se o array estiver ordenado em ordem crescente.

A diferença mais óbvia neste gráfico é onde as linhas se cruzam: a linha superior representa a resposta correta (opção

B) cruza a linha de traçado para uma das alternativas incorretas entre o segundo e o terceiro quartis. A linha também começa bastante baixa: menos de 80% dos alunos, mesmo no primeiro quartil, escolheram a resposta correta.

Esta questão distingue os alunos com melhor e pior desempenho: os quartis 1 e 2, por um lado, e os quartis 3 e 4, por outro. Além disso, revela equívocos comuns até mesmo entre alguns dos alunos com melhor desempenho.

Alunos de todos os níveis descartaram facilmente a alternativa C, que sempre retorna falso. Eles também descartaram as alternativas D e E, que retornam verdadeiro quando o array não está ordenado. Os alunos de melhor desempenho, no entanto, escolheram a resposta correta (B) em quase 75% das vezes, os alunos do segundo quartil escolheram a alternativa A quase tão frequentemente quanto a resposta correta, e os alunos do terceiro e quarto quartis preferiram a alternativa A à resposta correta.

O que explica a popularidade da alternativa A nesta questão? Mesmo os melhores alunos escolheram a alternativa A em mais de 20% das vezes. Na verdade, essa foi a alternativa mais eficaz em todas as questões para os melhores alunos.

As transcrições sugerem que os alunos foram induzidos ao erro devido a preconceitos sobre o significado de uma instrução `return` dentro de um laço. A alternativa B, a resposta correta, possui uma instrução `return` dentro de um laço; a alternativa A não. Os alunos que escolheram a alternativa A nem sempre ficaram satisfeitos com ela, mas consideraram-na "menos ruim" do que a resposta correta, B.

Encontramos três interpretações:

1. Quando a instrução `return` é encontrada, ela retorna um valor imediatamente e encerra o programa (interpretação correta);
2. Uma instrução `return`

dentro de um laço retorna múltiplos valores;

3. Uma instrução `return` dentro de um laço define um valor de retorno a cada iteração do laço e retorna seu valor final.

A segunda interpretação foi indicada por comentários como "No final, só pode retornar um verdadeiro ou falso. Não muitos." A terceira interpretação foi indicada por comentários como "você retorna falso e depois retorna verdadeiro, o que não está certo", "sempre retorna verdadeiro" e "mesmo que retorne falso, ainda assim retornará verdadeiro todas as vezes". Alguns alunos comentaram sobre a falta de um indicador: "sem identificadores", "talvez a alternativa B esteja errada porque você não está inicializando nenhuma variável".

Os problemas com instruções `return` dentro de um laço também explicam a diferença entre as alternativas D e E. Como mostrado no gráfico, D é preferível a E, embora ambas apresentem a mesma falha lógica (ambas retornam verdadeiro quando o array não está ordenado). D define um indicador, enquanto E possui uma instrução `return` dentro de um laço.

Por fim, considere as questões 11 e 12. Ambas as questões estão relacionadas ao mesmo tópico, e a questão 12 dá continuidade à questão 11. Aqui está o gráfico da questão 11:

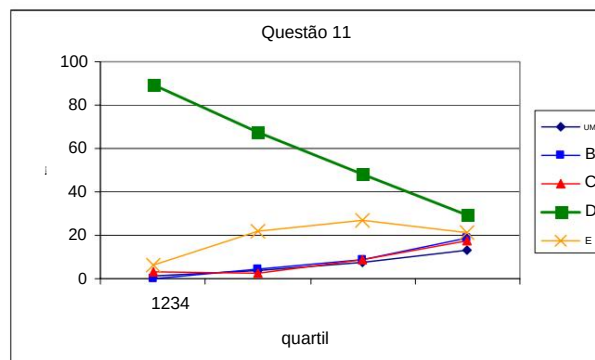


Figura B9: Respostas dos alunos à questão 11, por quartis.

Como pode ser observado no gráfico, esta foi uma questão mais fácil para os alunos. Ela envolve um array de números inteiros, ordenados em ordem crescente, com um "0" como último elemento, servindo como indicador do fim do array. O código foi desenvolvido para localizar a posição nesse array onde um novo elemento específico deve ser inserido. Os alunos foram solicitados a escolher a condição de laço correta para preencher uma lacuna no código.

As opções se concentraram em duas coisas:

1. a diferença entre um índice (pos) em um array e o elemento nesse índice (s[pos]).
2. a lógica da busca. Não podemos parar a busca até encontrarmos um elemento maior que o valor desejado. A condição correta é `while (s[pos] < e)`, e não `while (s[pos] != e)`.

Assim como nas questões de código fixo, esta também discrimina entre os três quartis superiores, por um lado, e os alunos com pior desempenho, por outro. Os alunos com melhor desempenho tiveram um excelente resultado. Raramente escolheram alguma das alternativas incorretas, embora, quando o fizeram, demonstraram uma ligeira preferência pela alternativa E. Os quartis 2 e 3 tiveram um desempenho inferior ao do quartil 1, mas ainda assim apresentaram uma forte preferência pela resposta correta (D). Quando escolheram uma resposta errada, preferiram a alternativa E a qualquer outra. Em outras palavras, não tiveram problemas em distinguir entre pos e s[pos] (indicado pela rejeição das alternativas AC), mas às vezes erraram na lógica de busca (indicado pela escolha da alternativa E).

Os alunos do quarto quartil são muito mais propensos a escolher as alternativas incorretas AC do que os alunos dos quartis 1 a 3, e menos propensos a escolher a resposta correta. Em outras palavras, eles são mais propensos a se confundir com a diferença.

entre pos e s[pos] do que os alunos nos outros quartis.

Finalmente, aqui está o gráfico de linhas traçadas para a Questão 12:

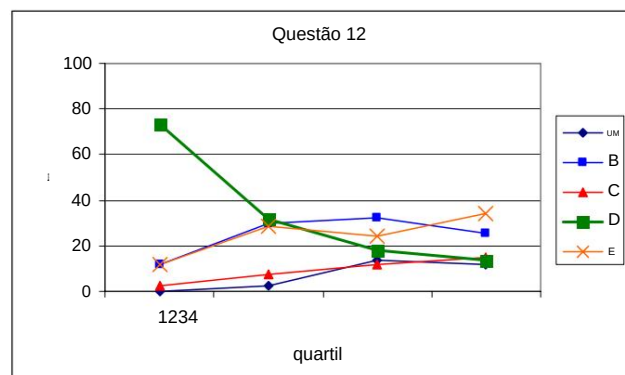


Figura B10: Respostas dos alunos à questão 12, por quartis.

Essa foi a questão mais difícil, e seu gráfico é o mais complexo. A questão 11 (assim como as questões de código fixo) separa os alunos com dificuldades dos demais; a questão 12 separa os melhores dos alunos medianos.

Esta questão envolve um único array de inteiros ordenado em ordem crescente, com um 0 após o último elemento do array, como na Questão 11. O código a ser escrito aqui deve remover um elemento na posição dada do array e deslocar os elementos restantes para a esquerda para preencher o espaço vazio. Assim, se removermos o 7 do array {1,3,5,7,9,0}, o resultado deverá ser {1,3,5,9,0,0}.

As opções se concentram em dois pontos:

1. Quando a instrução de atribuição $x=y$ é executada, ela é o valor de x que é alterado.
2. Dentro do loop, o índice do array é incrementado primeiro.

Alunos que estão confusos sobre a direção do operador de atribuição, mas percebem que o índice é incrementado primeiro, escolheriam a alternativa C. Alunos que compreendem bem o operador de atribuição, mas não percebem que o índice do array é incrementado primeiro, escolheriam a alternativa B. Este é o código esperado e idiomático. A alternativa A contém ambos os erros. D é a resposta correta e E é "nenhuma das alternativas acima". Esta é a única questão que continha a opção "nenhuma das alternativas acima".

Os melhores alunos escolheram a resposta correta em mais de 70% das vezes e, quando não a escolheram, tiveram a mesma probabilidade de optar pela alternativa B (a resposta idiomática) e pela alternativa E (nenhuma das alternativas acima). Quase nunca selecionaram A ou C.

Nos quartis 2 a 4, a resposta correta deixa de ser a mais popular. No quartil 2, ela está aproximadamente empatada com as alternativas B e E; no quartil 3, as alternativas B e E são mais populares que a resposta correta, mas B (a resposta idiomática) ainda é a mais popular; e no

No quarto quartil, a alternativa E (nenhuma das alternativas acima) é a escolha mais popular.

Ao que parece, a maioria dos alunos entende o operador de atribuição, mas não é incomum que eles assumam que o índice é incrementado no local usual, no final do corpo do laço.

Esta questão em si não parece muito mais difícil do que as outras. Alguns alunos podem ter desistido por não terem entendido a questão 11 (embora esta questão provavelmente pudesse ser respondida sem ler a questão 11). Devido à continuação, a questão 12 parece exigir mais leitura do que as demais. Vários alunos foram induzidos ao erro pelo fato de o índice do array ser incrementado no início do corpo do loop. Notavelmente, esta é a única questão que contém uma opção "nenhuma das anteriores", e isso pode ter induzido os alunos ao erro: o simples fato de não terem visto essa opção antes pode torná-la mais significativa. E pode ser simplesmente que os alunos estivessem cansados a essa altura da prova. Pelo menos uma das transcrições indicou que, a essa altura, o aluno havia perdido a concentração e estava simplesmente chutando.